

COMSATS University Islamabad (CUI)

Software Design Description
(SDS DOCUMENT)

for

PodVerse

Version 1.0

By

Rabeea Chughtai **CIIT/SP22-BCS-079/ISB**

Sadia Imran **CIIT/SP22-BCS-087/ISB**

Supervisor

Ms. Saadia Maqbool

Bachelor of Science in Computer Science (2022-2026)

Table of Contents

Revision History ..... **iii**

Application Evaluation History ..... **iv**

1. Introduction **1**

1.1 Scope 1

2. Design Methodology and Software Process Model **5**

2.1 Design Methodology: 5

2.2 Software Process Model: 5

3. System Overview **6**

3.1 Architectural Design 6

4. Design Models **9**

4.1 Activity Diagrams 9

4.2 Class Diagram 23

4.3 Sequence Diagrams 24

4.4 State Transition Diagrams 33

5. Data Design **35**

5.1 Data Dictionary 35

6. Implementation **44**

6.1 Algorithm 44

6.2 External APIs/SDKs 45

6.3 User Interface 46

6.4 Deployment 46

7. Testing and Evaluation **47**

7.1 Unit Testing 47

7.2 Functional Testing 51

7.3 Business Rules Testing 55

7.4 Integration Testing 56

8. References **59**

Revision History

Name	Date	Reason for changes	Version

Application Evaluation History

Comments (by committee) ***include the ones given at scope time both in doc and presentation**	**Action Taken**
Scope should be increases, add major modules e.g. a usecase where these podcasts can be used. At the moment the scope is not enough for 2 students with pretrained models/api usage.	• Added Social Media Integration for podcast sharing and engagement. • Introduced Podcast Summary Feature for better accessibility and usability. • Integrated Admin Panel & Analytics for user/content management and insights. • Expanded Audience Engagement with messaging, notifications, and user interactions. • Enhanced AI Video Podcast Creation with avatars, and visual elements
1. Students lack in knowledge of Agent/AI Agent. 2. Agent creation is unknown. 3. Tools' usage and purpose are not clear. 4. Mockups are betrayal. 5. Students' effort is unclear and unjustified with time required. 6. Prompt engineering and voice to text generation are unclear to students. 7. Lack of existing study	

Supervised by

Supervisor's Name

Signature_____

1. Introduction

PodVerse is an AI-powered podcast creation platform developed to simplify and modernize the podcast production journey for small business owners, entrepreneurs, and content creators. By eliminating technical barriers such as high production costs, complex editing processes, and challenges in finding credible guests, PodVerse enables creators to produce engaging, high-quality podcasts effortlessly. The system integrates advanced technologies like voice cloning, real-time AI interactions, and customizable avatars, providing users with a centralized solution for planning, recording, editing, and publishing their podcast content. With a user-friendly interface and modules tailored for automation and personalization, PodVerse ensures seamless experience, empowering creators to build consistent content and strengthen audience engagement.

1.1 Scope

The scope of PodVerse is covered under the following modules:

1.1.1 Module 1: User Authentication & Profile Management

This module manages authentication, profile customization, and dashboard access for secure login and content engagement.
FE-1: Account creation via email with verification.
FE-2: Secure login via email/password.
FE-3: Password recovery via email verification.
FE-4: Secure Logout.
FE-5: Creator's dashboard for sample podcasts and AI Speaker management.
FE-6: View and edit profiles.
FE-7: Activity feeds with updates from followed creators.

1.1.2 Module 2: Voice Cloning

This module allows users to create and manage voice samples for cloning.
FE-1: Enabling the recording or upload of voice samples for cloning.
FE-2: Allowing users to preview voice samples to check on the quality.
FE-3: Allow users to re-record or re-upload if the sample is not as desired.
FE-4: Allow the availability of a library with voices.

1.1.3 Module 3: Knowledge Base Setup

This module facilitates the setup and management of a knowledge base for retrieval.
FE-1: Allow users to upload data in PDFs, URLs, CSV, or JSON.
FE-2: Data is cleaned, processed, and split into meaningful chunks for easy handling.
FE-3: Tag chunks with metadata and link to original sources.
FE-4: Store chunks in a database and generate vector embeddings for better retrieval.

1.1.4 Module 4: RAG Retrieval

This module enhances the retrieval of accurate and relevant information from the knowledge base.
FE-1: Convert queries into vectors to determine purpose.
FE-2: Match vectors with knowledge base data for accurate responses.
FE-3: Re-rank data by relevance for better answers.
FE-4: Provides fact-based correct answers for podcasts.
FE-5: Optimized for large datasets without loss of performance.
FE-6: Allow feedback to enhance AI Speaker response quality.

1.1.5 Module 5: AI Speaker Customization

This module allows users to personalize their AI Speaker.
FE-1: Users can assign a custom name to their AI Speaker.
FE-2: Users can choose from various voice options to suit their podcast.
FE-3: Users can select or upload an avatar to visually represent their AI Speaker.

1.1.6 Module 6: Podcast Audio Creation

This module provides tools for creating and publishing podcasts.
FE-1: Let users start dialogues with a customized AI Speaker.
FE-2: Conversations recorded for user and AI Speaker.
FE-3: Users allowed minimal editing to trim and splice to rearrange things.
FE-4: Allow users the ability to preview edited podcasts for flow and polish.
FE-5: Export podcasts in MP3/WAV.
FE-6: Save recordings for future use.

1.1.7 Module 7: AI Host

This module provides an AI host to manage the flow of podcasts.
FE-1: AI Host presents the flow of podcasts, introduces the episodes and closes with a summary.
FE-2: Tracks podcast time and gives reminders for the schedule.
FE-3: Facilitates smooth transitions between guests or topics.
FE-4: Provides summary prompts to summarize key points for clarity and engagement.

1.1.8 Module 8: AI Guest

This module provides an AI guest to enhance podcast discussions.
FE-1: AI Guest responds to real-time queries using dynamic responses.
FE-2: AI Guest shares appropriate stories to add depth to the discussion.
FE-3: AI Guest gives back comments/reactions to facilitate engagement.

1.1.9 Module 9: Podcast Video Creation & Publishing

This module enables the creation of video content for podcasts, and AI-generated text summaries.
FE-1: Users select avatars for each of the speakers.
FE-2: The system syncs based on the audio of the podcast.
FE-3: Users can choose from the pre-defined templates and visuals for the video.
FE-4: Pre-designed split-screen video template with two avatars.
FE-5: Exports the final video in MP4 format or publish the final video.
FE-6: Allow creators to rate podcast sessions.
FE-7: Provide written feedback on conversation skills, voice clarity, and AI accuracy.

1.1.10 Module 10: Subscription & Payment

This module manages subscription and payment processes.
FE-1: Let users access some free, limited content.
FE-2: After the limit, users are prompted to subscribe for continued access.
FE-3: Secure payment gateway integration supporting credit/debit cards.
FE-4: User can control subscriptions, see payment history, change methods.
FE-5: Give reminders before expiration with options of auto-renewal.

1.1.11 Module 11: FAQs Chatbot

This module provides a chatbot to answer frequently asked questions.
FE-1: Users can ask about podcasting, AI customization, or troubleshooting.
FE-2: The chatbot retrieves relevant answers from the FAQs and support materials.
FE-3: Provides concise, direct responses for quick problem resolution.

1.1.12 Module 12: Audience Engagement and Communication

This module enables audience interaction, providing a communication platform, and podcast feedback to enhance creator engagement.
FE-1: Notifications for creators about audience interactions.
FE-2: Highlighting top creators and trending content.
FE-3: Integrated communication platform for user-to-user messaging and calls.
FE-4: Generate and display a text-based summary of podcast episodes.

1.1.13 Module 13: Admin Panel & Analytics

This module provides administrative functionalities, including user management, content moderation, analytics, and system configuration.
FE-1: Allow admin to view, search, deactivate, reactivate, or delete user accounts
FE-2: Enable admin to review reported podcasts and comments.
FE-3: Display live statistics on active users, total podcasts created, trending content, user engagement heatmaps, and demographics.
FE-4: Allow admin to track active subscriptions, process refunds, and manage payment disputes.
FE-5: Provide insights into user engagement, podcast performance, and trending topics.

2. Design Methodology and Software Process Model

2.1 Design Methodology:

In developing PodVerse, we've chosen an object-oriented approach.

2.1.1 Rationale behind this approach:

Here are the main reasons for choosing this approach:

Modularity and Reusability:
For PodVerse, which has many features like user authentication and AI-driven podcast creation, having a modular structure is important. Each class can focus on a specific task, making the code easier to manage and less likely to have errors.

Efficiency in Development:
The object-oriented approach allows for rapid development. By breaking down the application into smaller, reusable classes, our team can work on different parts of the app at the same time.

Better Debugging and Testing:
Object-oriented programming allows for testing and debugging of individual classes separately. This means each part of PodVerse can be thoroughly tested before integration, reducing bugs and ensuring higher code quality.

2.2 Software Process Model:

In developing PodVerse, we are following the Incremental model.

2.2.1 Rationale behind this model:

Here are the main reasons for choosing this model:

Step-by-Step Progress:
The Incremental model allows the project to grow through iterative cycles. With each cycle, new features are added, tested, and refined. This ensures that the system is developed and tested in steps, enabling early delivery of core functionalities while allowing for improvements in later cycles.

Feedback Integration:
By delivering the system in steps, we can gather feedback from users at each stage. Early and continuous feedback helps identify any issues or areas for improvement, allowing us to refine and enhance the system iteratively.

Flexibility in Adding Features:
The incremental model provides flexibility in adding new features later. Modules like AI Host, AI Guest, and Podcast Video Creation can be developed and integrated one by one. This approach allows us to adapt to changing requirements.

Continuous Testing and Quality Assurance:
Each step is tested thoroughly before proceeding, allowing early detection and resolution of issues. This ensures a stable, reliable system and smooth user experience.

3. System Overview

PodVerse is an innovative platform designed to simplify podcast creation and production for small business owners, entrepreneurs, and content creators. It addresses common challenges such as high production costs, technical complexities, and securing credible guests. PodVerse enables users to create high-quality podcasts effortlessly, even without extensive technical skills. The platform offers features like voice cloning, editing, and publishing, making the production process seamless. With AI voices and language models, users can create engaging and dynamic podcasts without needing real hosts or guests. PodVerse is built to help creators produce consistent content, grow their brand, and engage effectively with their audience.

3.1 Architectural Design

The architectural design of PodVerse is modular, ensuring that each component of the system works together to achieve the overall functionality. This section provides a high-level overview of how the system's modules collaborate to deliver the desired features.

3.1.1 Box and Line Diagram

The following diagram shows the modules and their relationships with each other. This diagram is at a high level of abstraction and gives an overview of how different modules of the system interact to achieve the desired functionality.

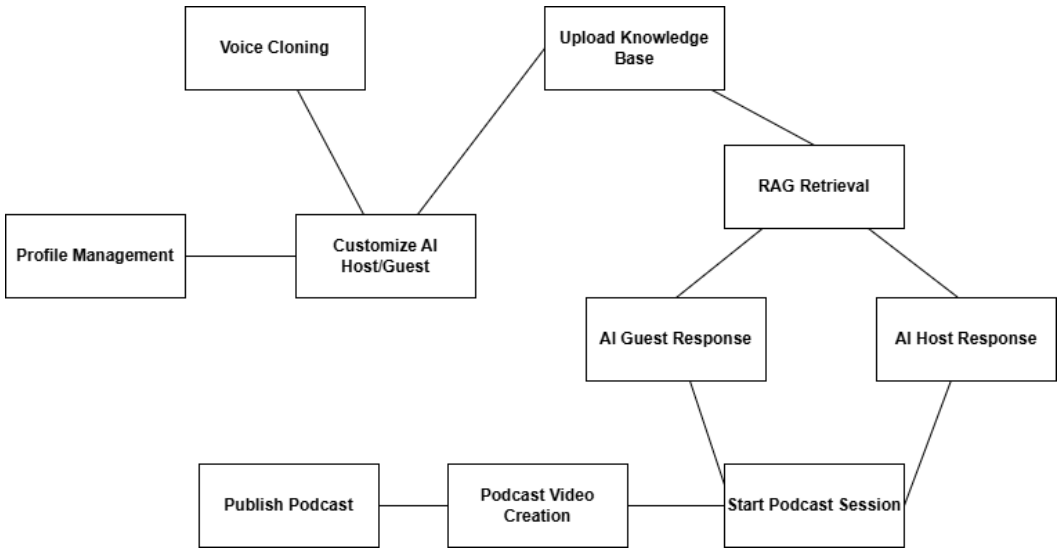


Figure 1: Box and Line Diagram of PodVerse

3.1.2 MVC Architecture

We have adopted a MVC Architecture for the PodVerse application. This architecture style emphasizes the separation of concerns and facilitates modularity.

Presentation Layer (View):

This layer includes the user interface components that interact with the users, such as AudioCreation_View, VideoCreation_View, and ProfileSetting_View. It captures user input and displays output, enabling smooth navigation and interaction within the system.

Application Layer (Controller):

Controllers like AudioCreation_Controller, VoiceCloning_Controller, and Auth_Controller handle user actions, process inputs, coordinate between the view and model, and manage application logic such as podcast creation and authentication.

Business Logic Layer (Model):

This layer contains core logic and data models such as User, Podcast, VoiceSample, and Rating. It manages processing rules, validations, AI interactions, and handles the overall behavior of the system.

AI Services & External Layer:

This layer integrates external services like the Voice Cloning Engine, LLMs, RAG Retrieval, and Subscription APIs. It has intelligent features such as AI guest responses, voice synthesis, and content recommendations.

Data Access Layer:

It abstracts database interactions and performs CRUD operations through repositories. It accesses data stored in relational databases, vector stores, and cloud storage to support both structured and unstructured data needs.

Database Layer:

This layer consists of the actual storage systems, including relational databases for core data, vector stores for AI embeddings, and cloud storage for media files like audio and video podcasts.

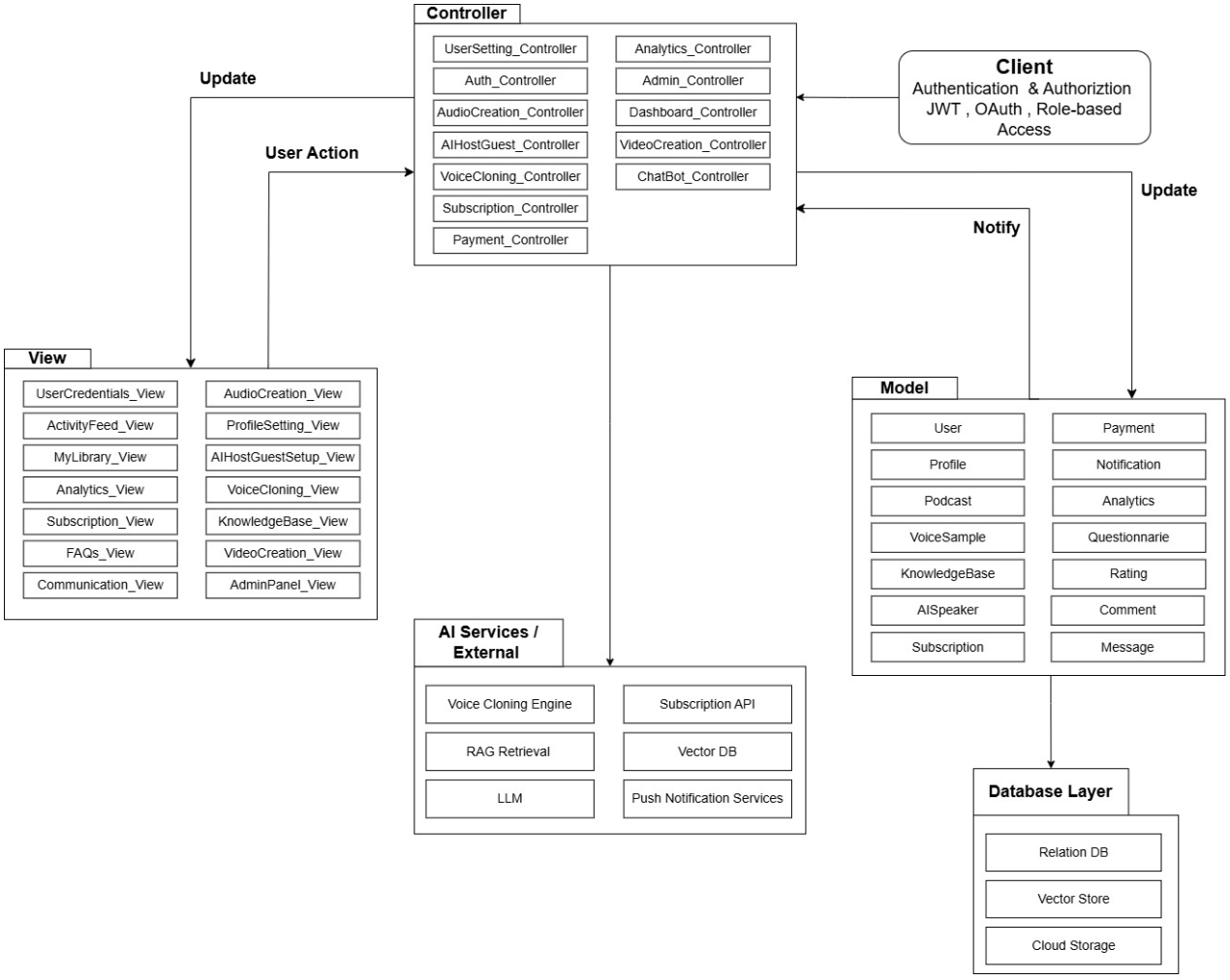


Figure 2: Architecture Diagram for PodVerse

4. Design Models

This section shows the design models for the system.

4.1 Activity Diagrams

4.1.1 Register Account

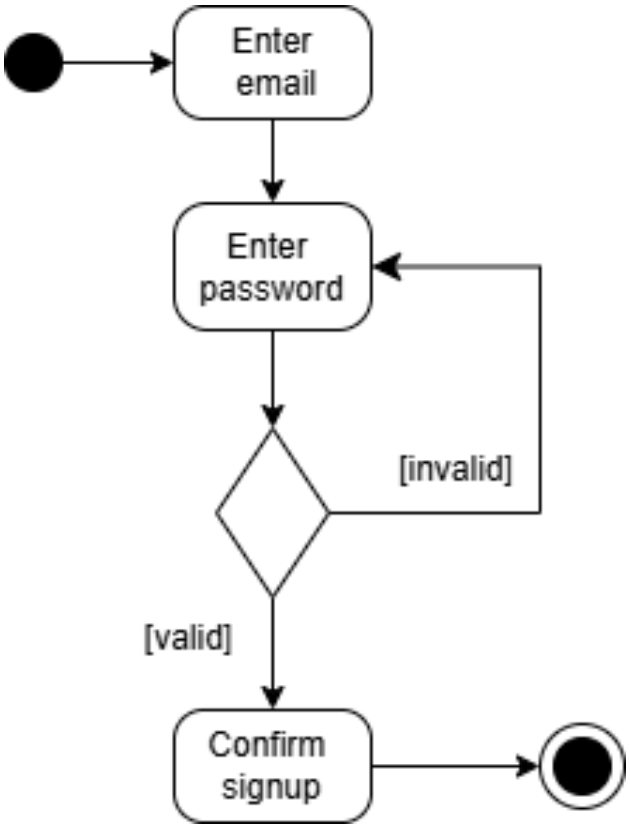


Figure 3: Activity Diagram for Sign up

4.1.2 Login

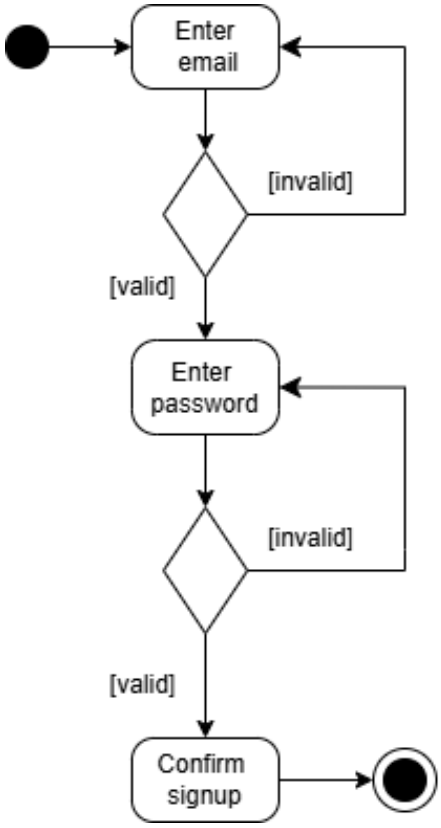


Figure 4: Activity Diagram for Sign in

4.1.3 Edit Profile

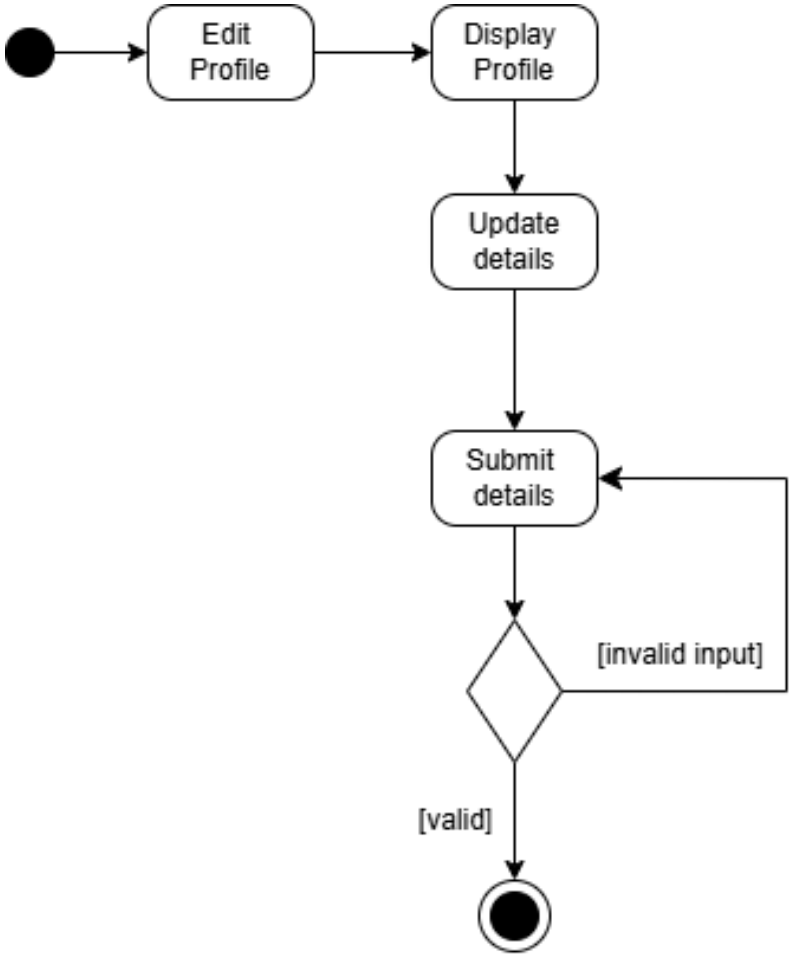


Figure 5: Activity Diagram for Edit Profile

4.1.4 Voice Cloning

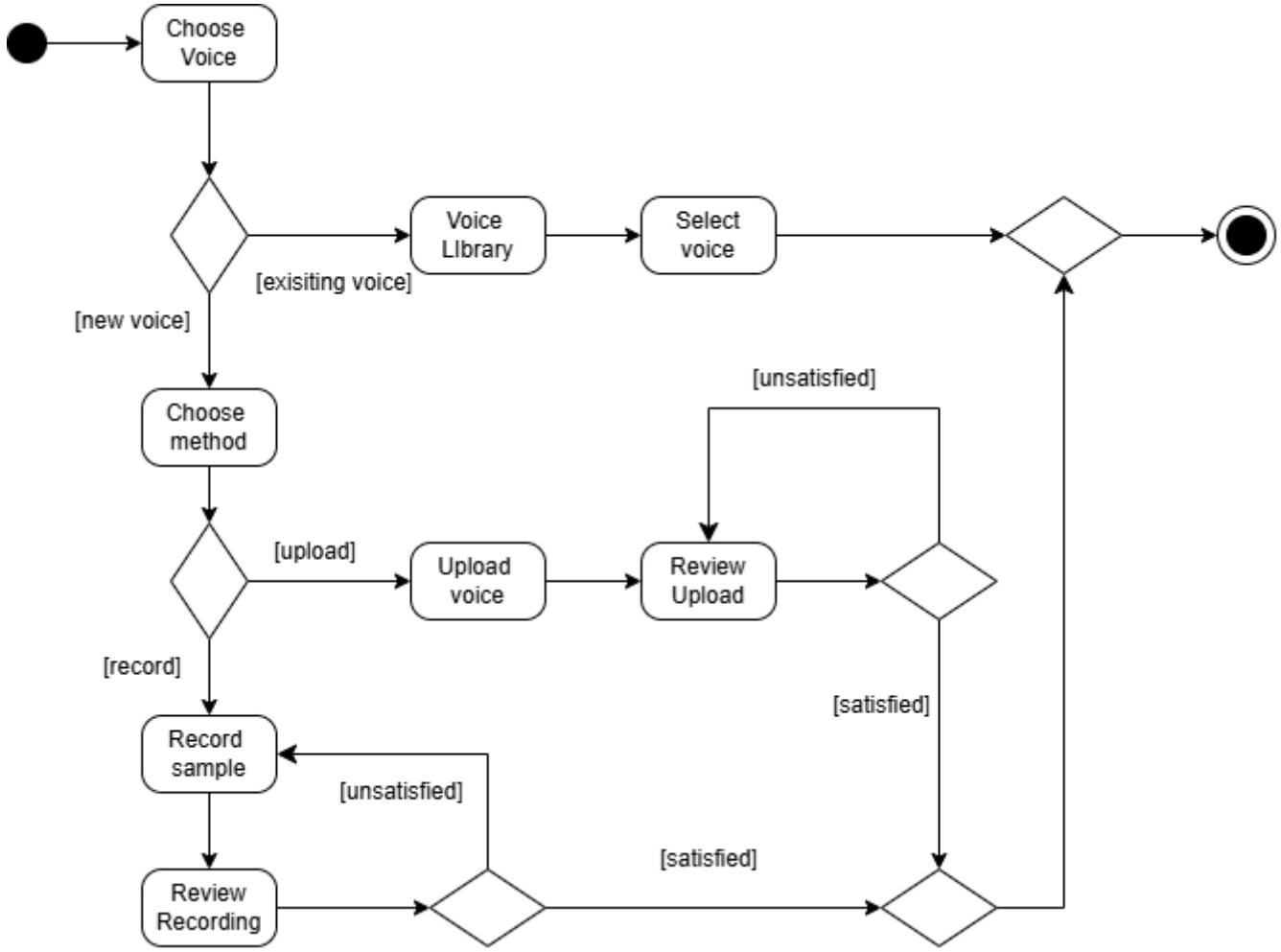


Figure 6: Activity Diagram for Voice Cloning

4.1.5 Knowledge Base Setup

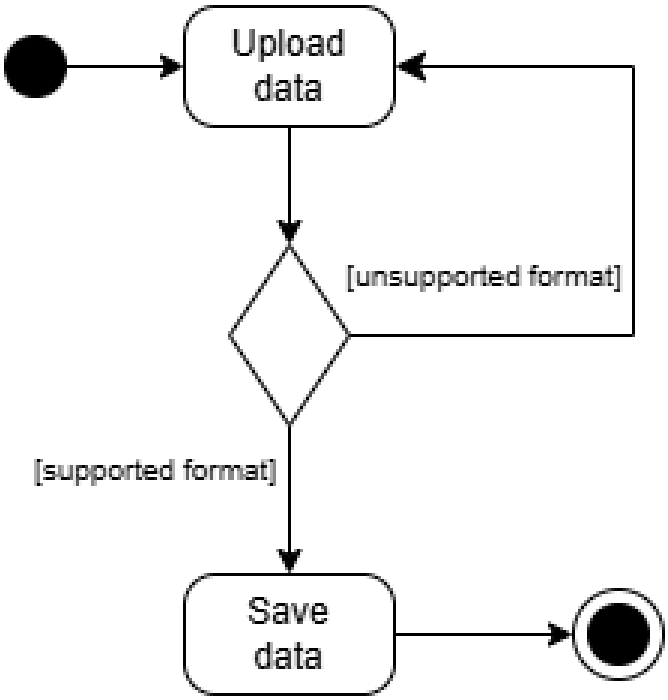


Figure 7: Activity Diagram for Knowledge Base Setup

4.1.6 AI Speaker Customization

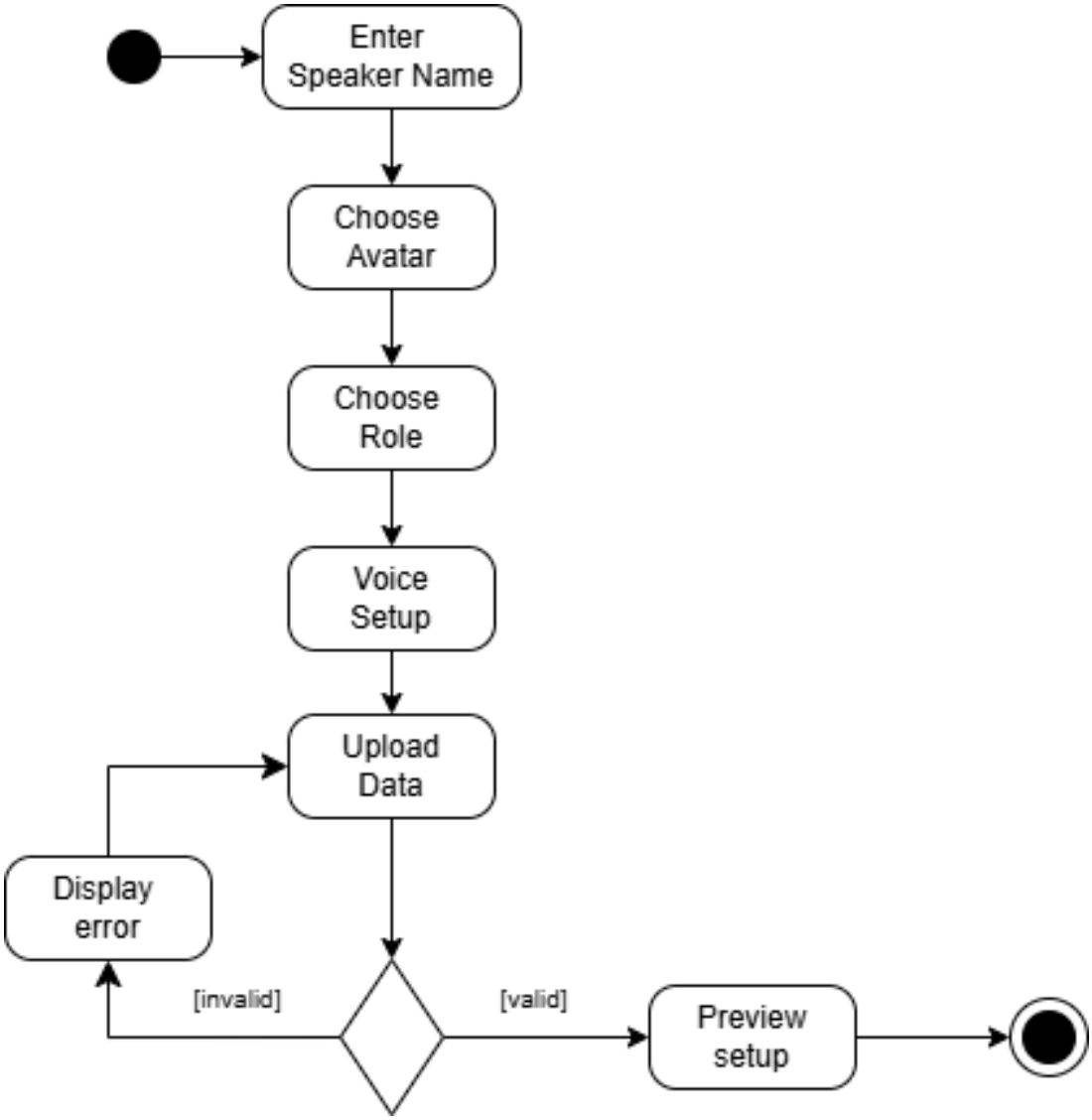


Figure 8: Activity Diagram for AI Speaker Customization

4.1.7 Podcast Audio Creation

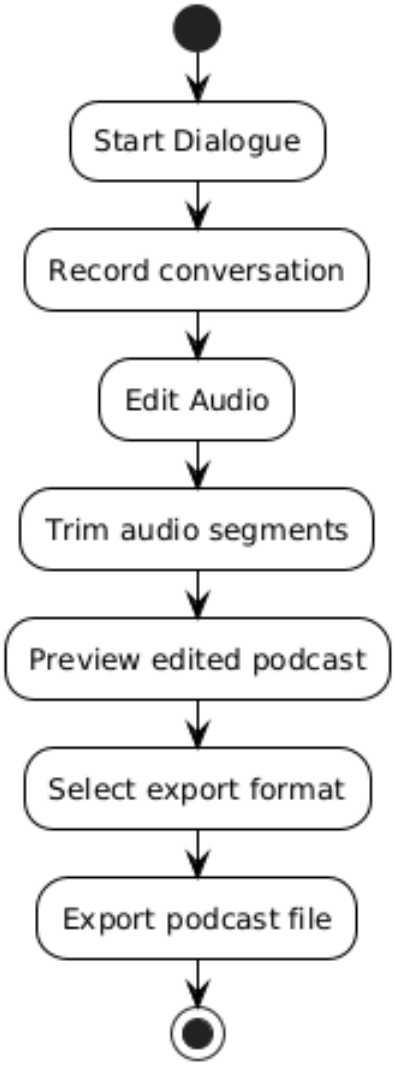


Figure 9: Activity Diagram for Podcast Audio Creation

4.1.8 Podcast Video Creation

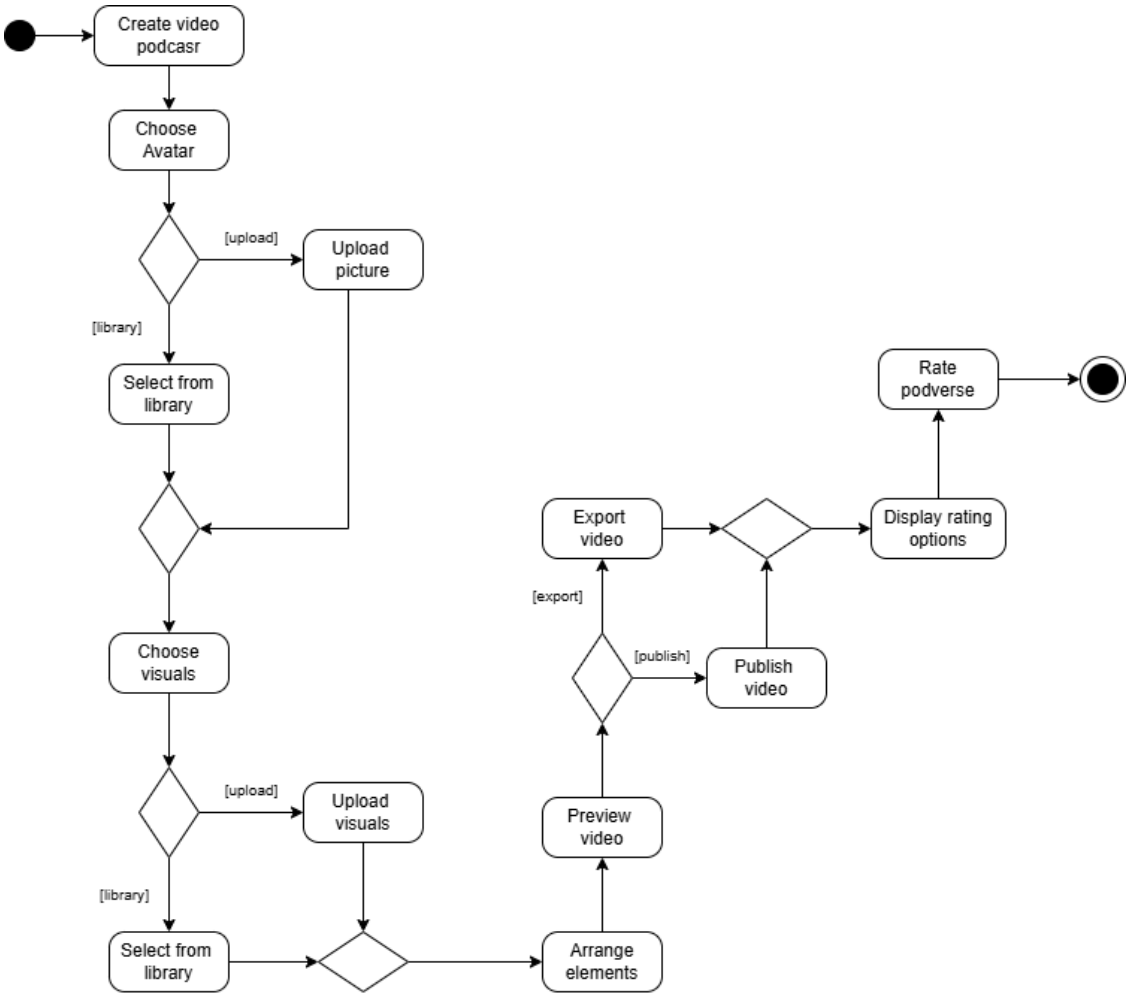


Figure 10: Activity Diagram for Podcast Video Creation

4.1.9 Request Podcast Summary

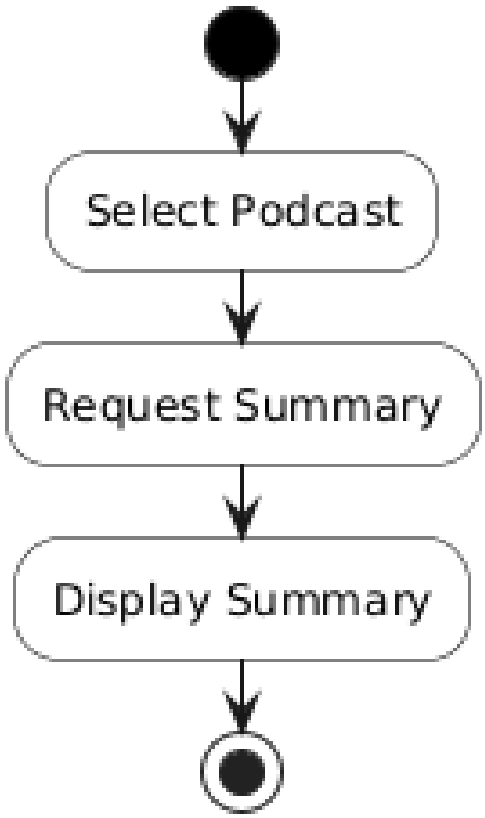


Figure 11: Activity Diagram for Podcast Summary

4.1.10 Subscription and Payment

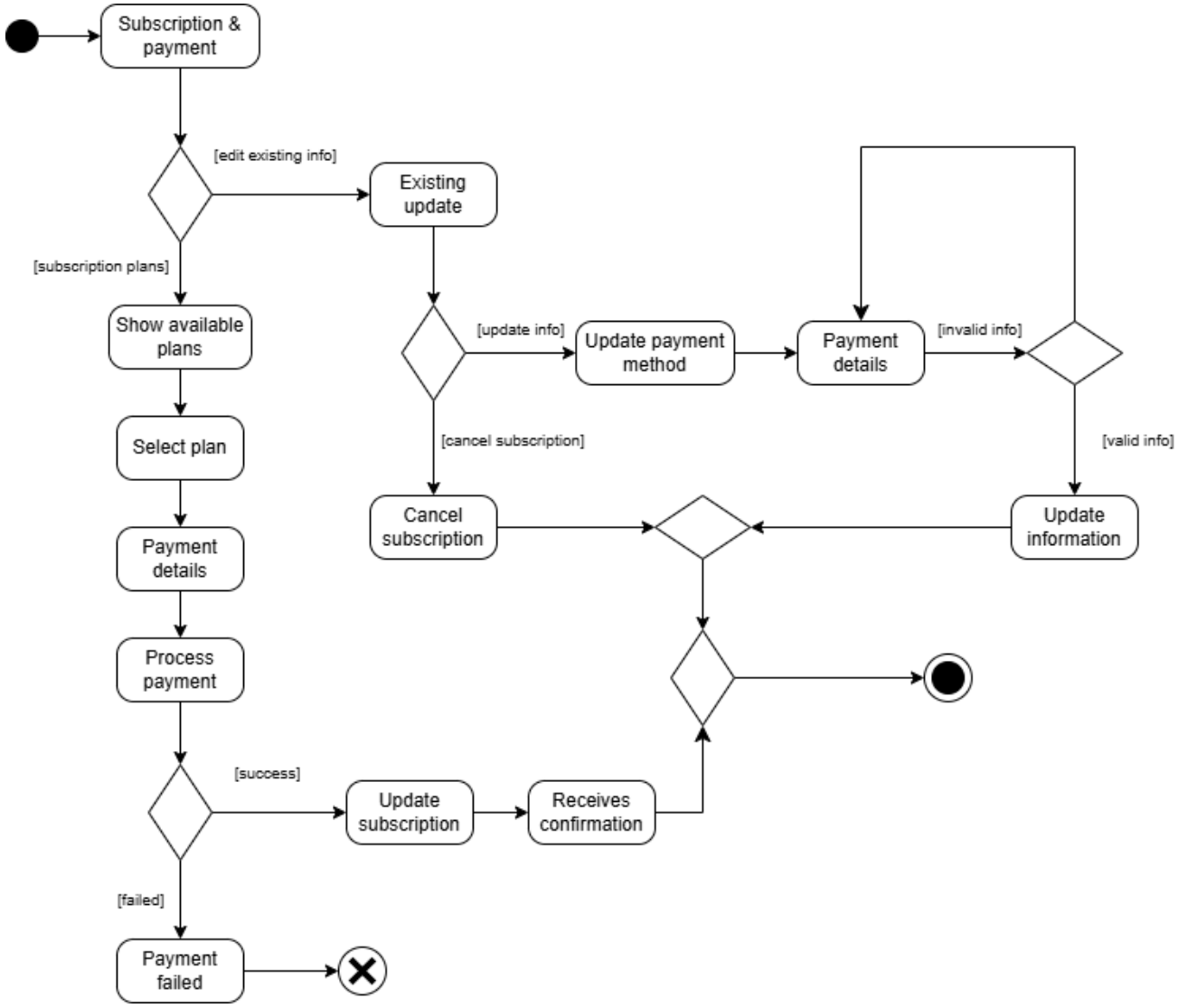


Figure 12: Activity Diagram for Subscription and Payment

4.1.11 FAQs Chatbot

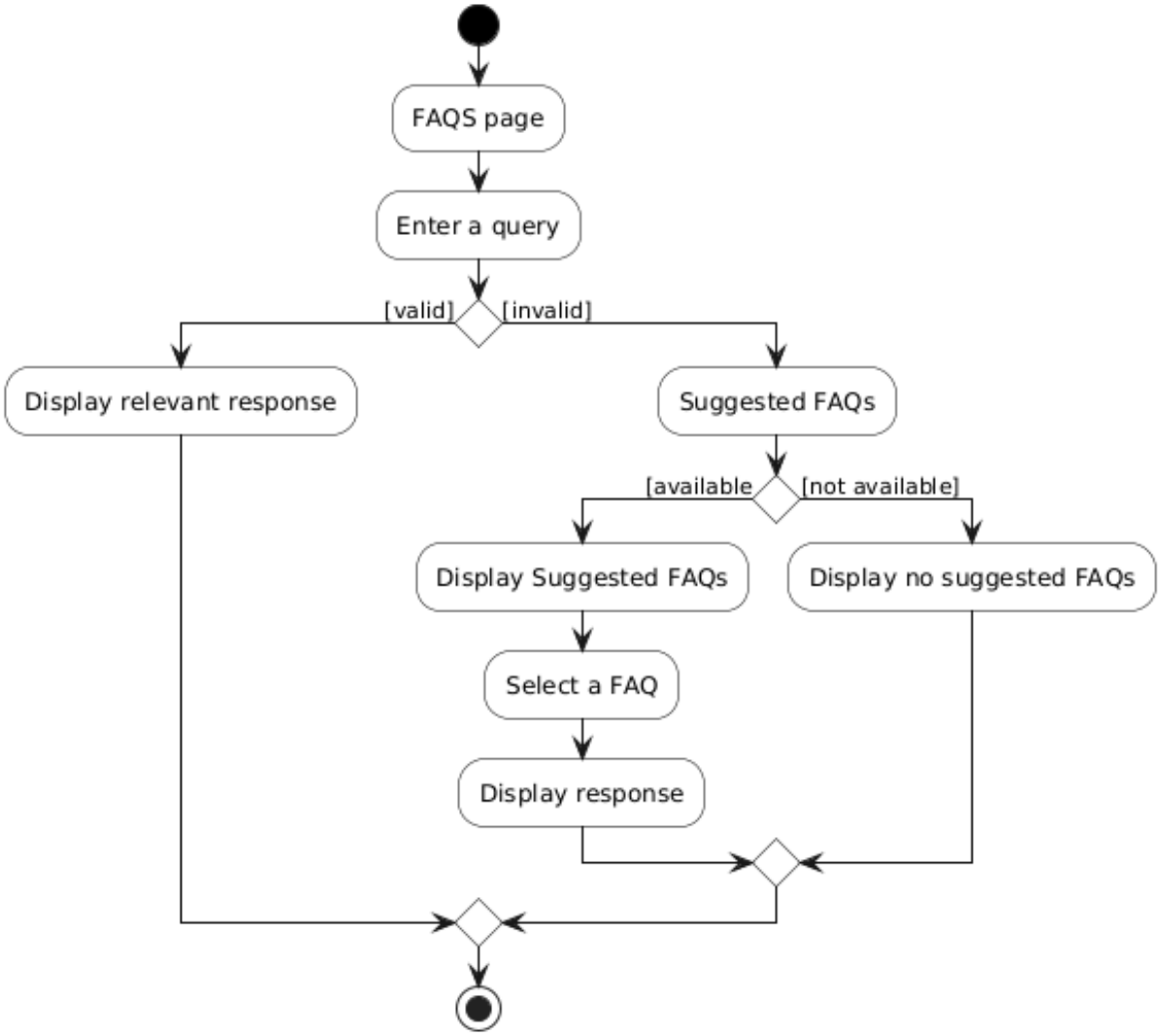


Figure 13: Activity Diagram for FAQs Chatbot

4.1.12 User-to-User Messaging and Calls

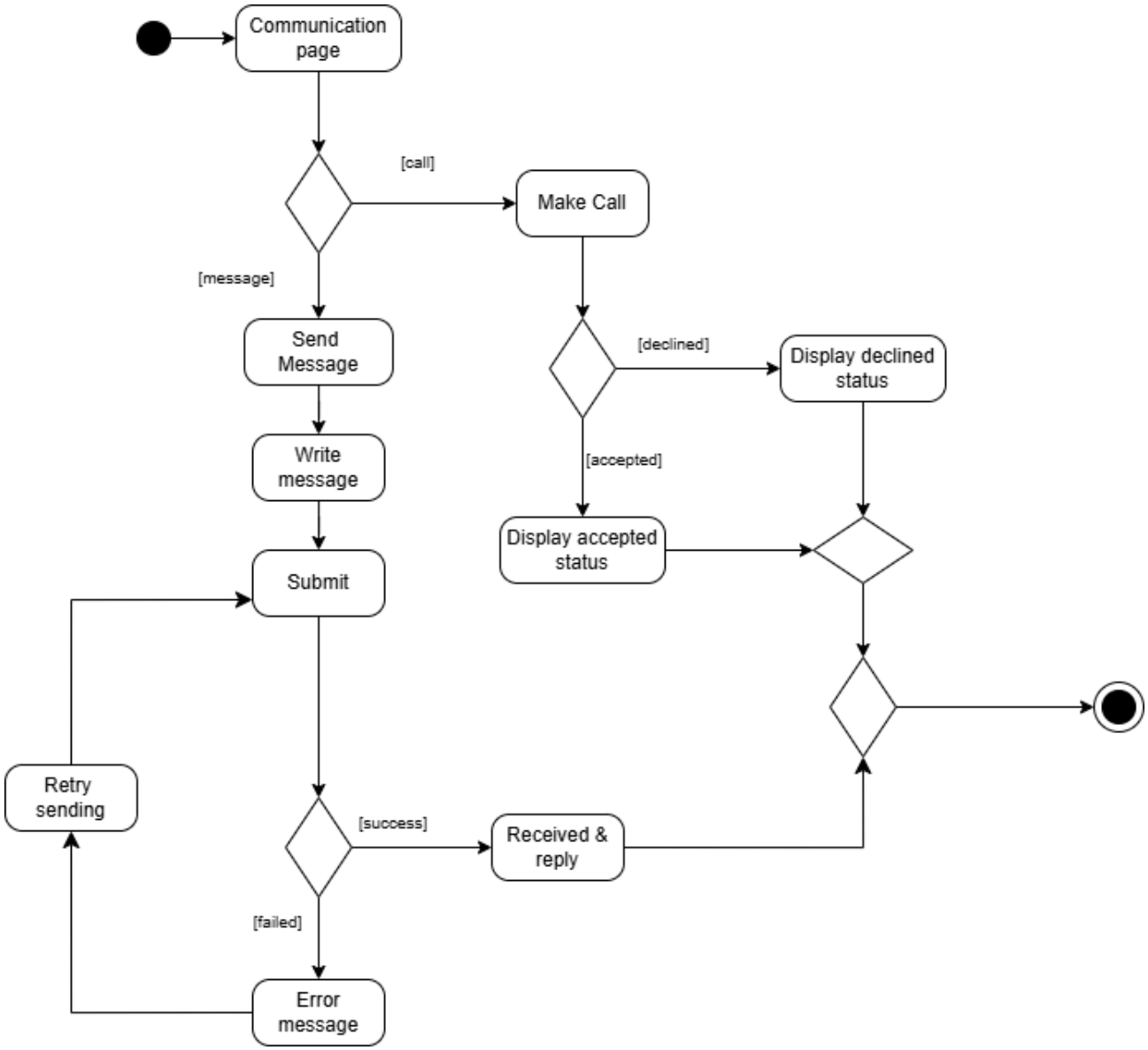


Figure 14: Activity Diagram for User-to-User Messaging and Calls

4.1.13 Manage Users and Subscriptions

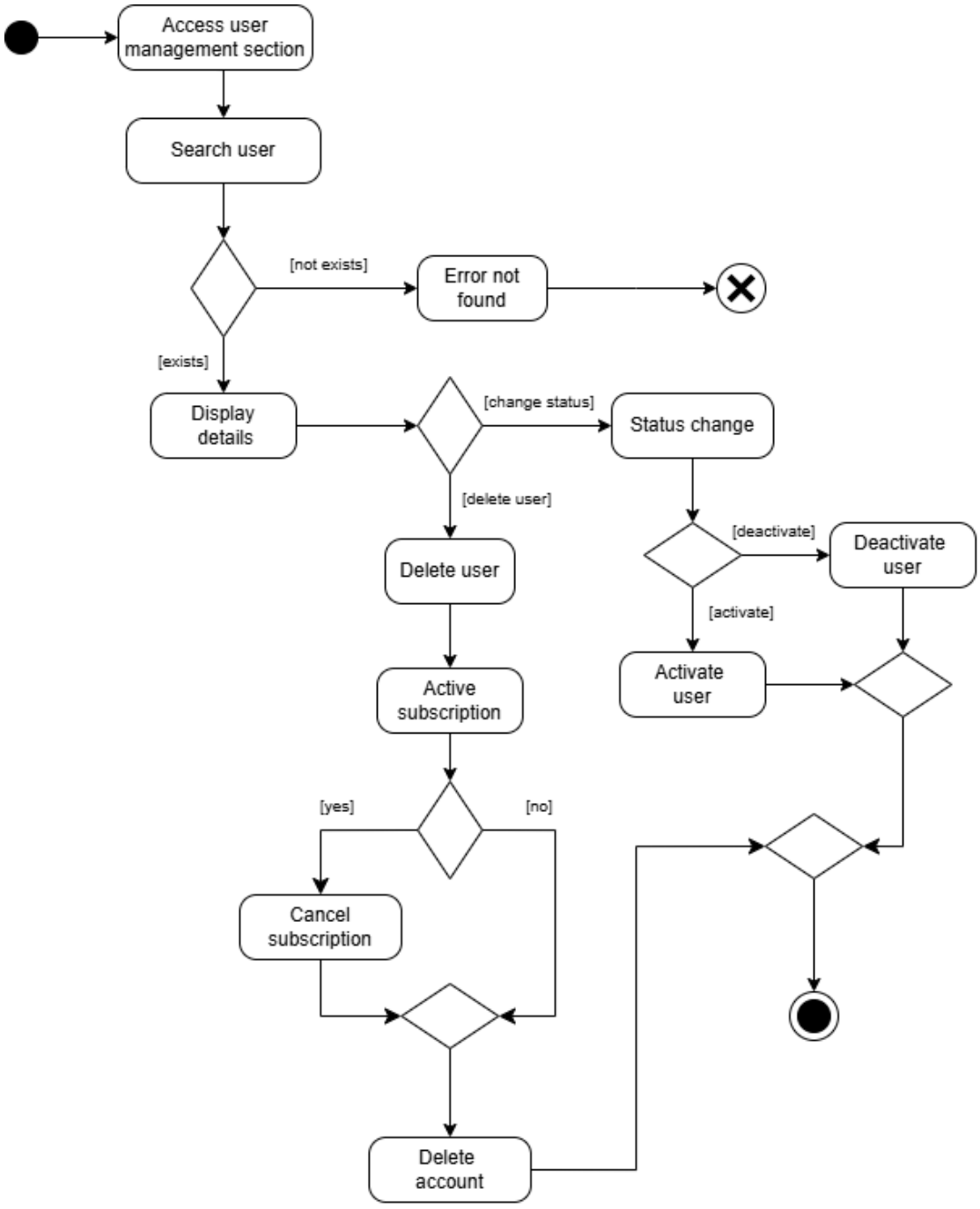


Figure 15: Activity Diagram for Managing Users and Subscriptions

4.1.14 Logout

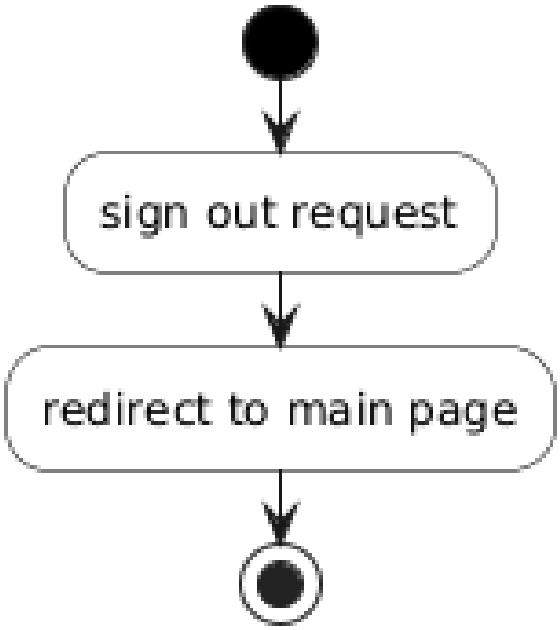


Figure 16: Activity Diagram for Sign out

4.2 Class Diagram

The following are the class entities for PodVerse:

1 User
2 Profile
3 Podcast
4 Episode
5 AIHost
6 AIGuest
7 VoiceSample
8 KnowledgeBase
9 Chunk
10 Message
11 Comment
12 Rating
13 Subscription
14 Payment
15 Notification
16 FAQsChatbot
17 FAQEntry
18 AISpeaker
19 Admin

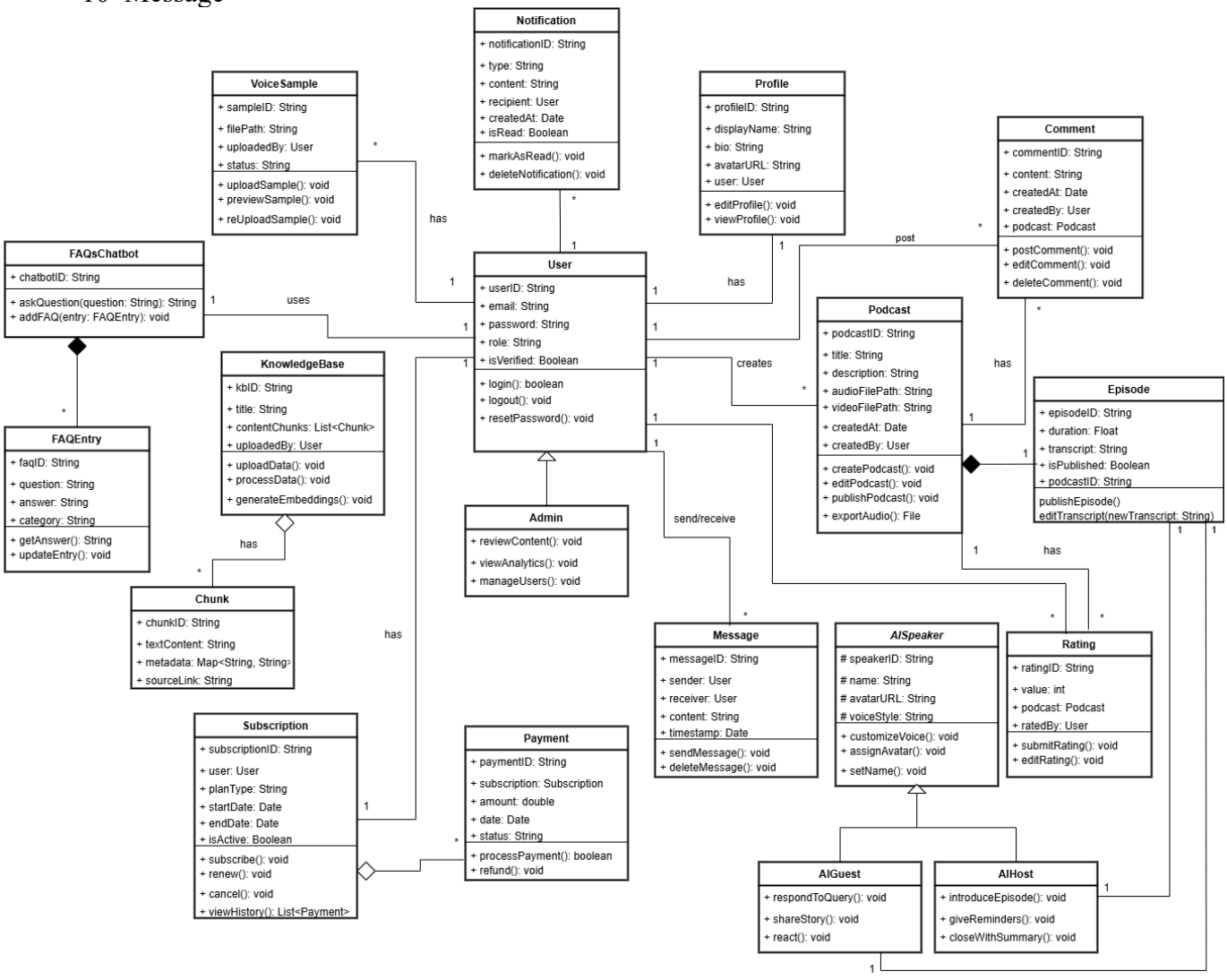


Figure 17: Class Diagram for PodVerse

4.3 Sequence Diagrams

Following are the sequence diagrams of PodVerse:

4.3.1 Module 1: User Authentication

Sequence diagram of module 1.

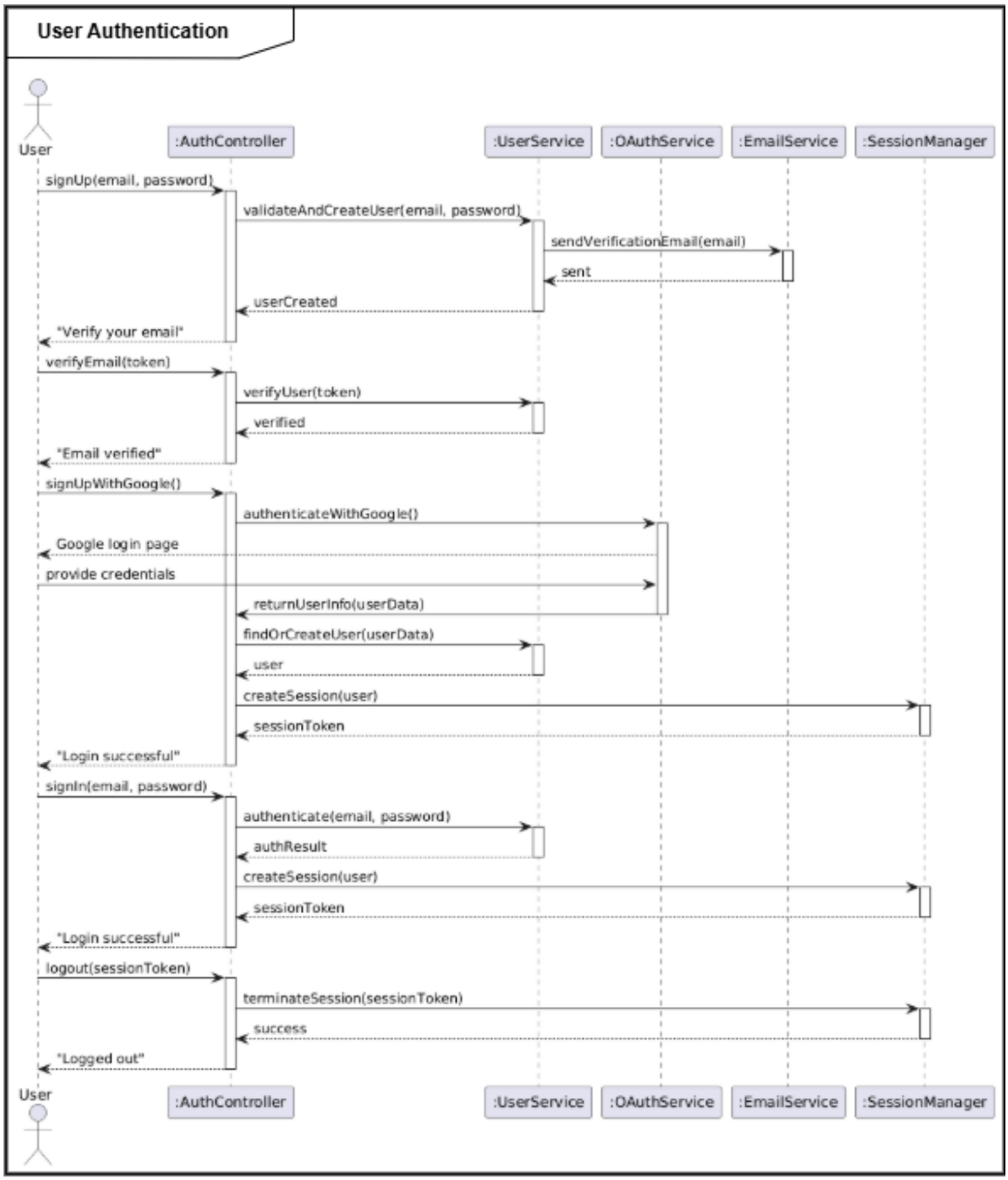


Figure 18: Sequence diagram of module 1

4.3.2 Module 2: Voice Cloning

Sequence diagram of module 2.

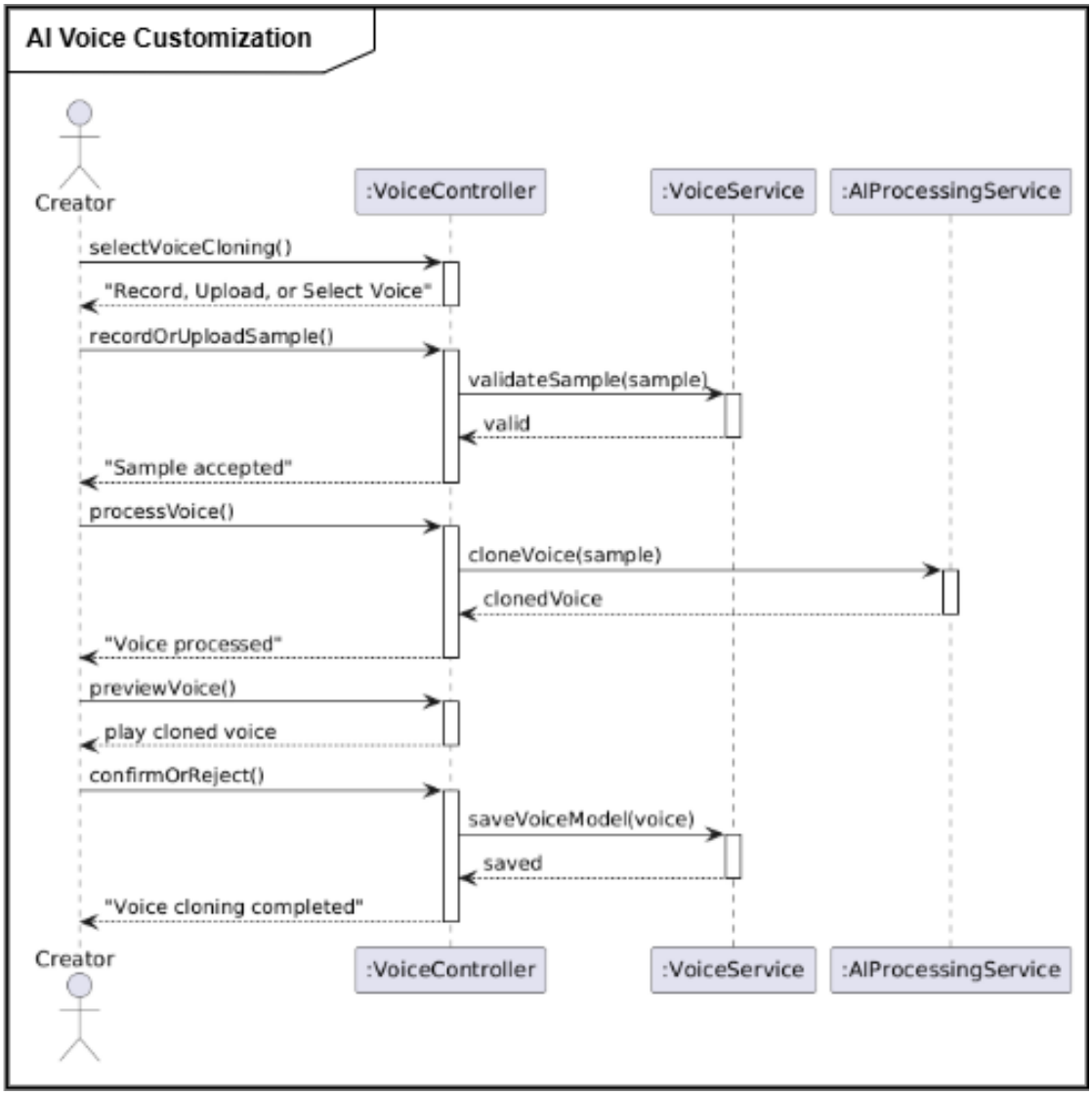


Figure 19: Sequence diagram of module 2

4.3.3 Module 3: Knowledge Base Setup

Sequence diagram of module 3.

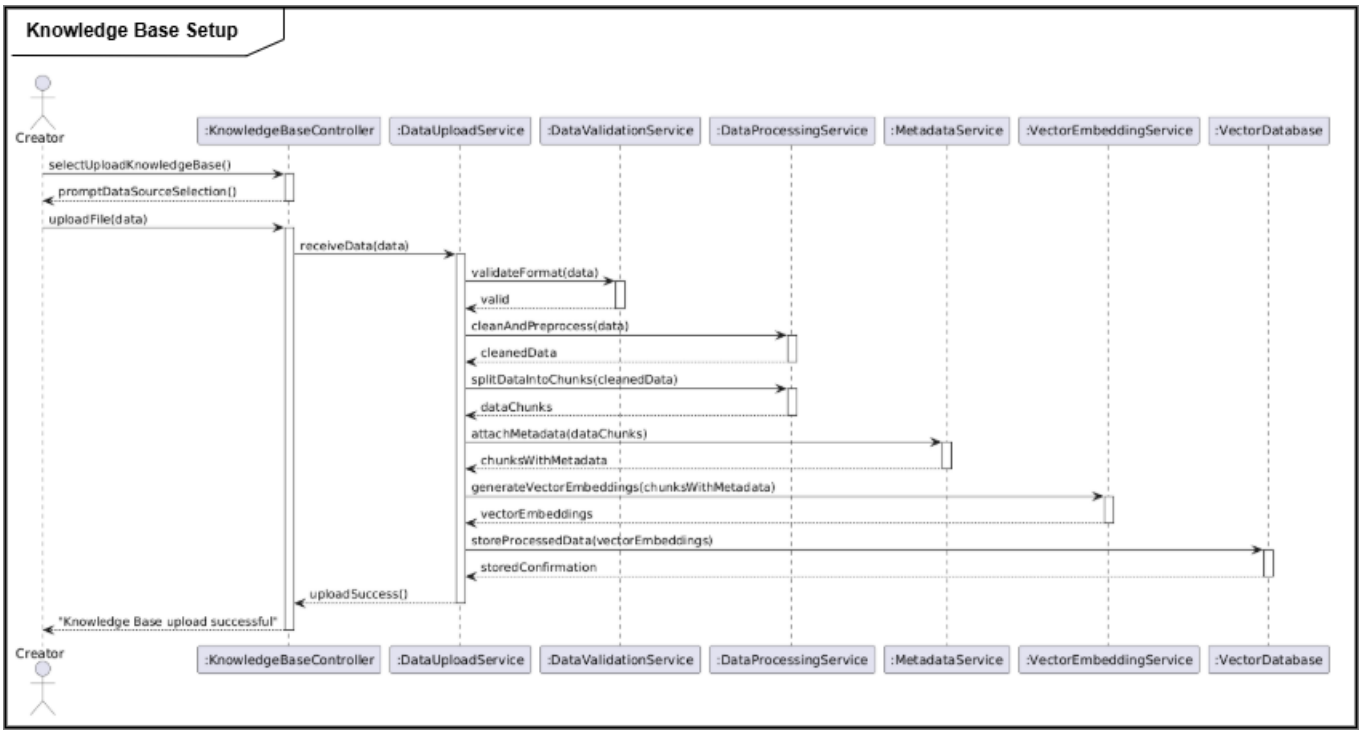


Figure 20: Sequence diagram of module 3

4.3.4 Module 4: RAG Retrieval

Sequence diagram of module 4.

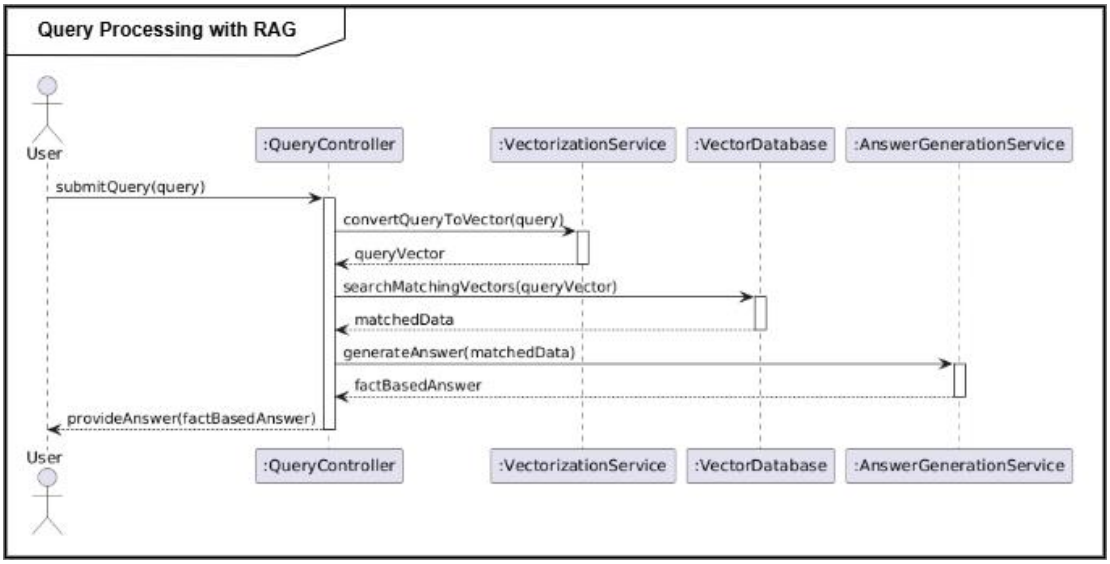


Figure 21: Sequence diagram of module 4

4.3.5 Module 6: Podcast Audio Creation

Sequence diagram of module 6.

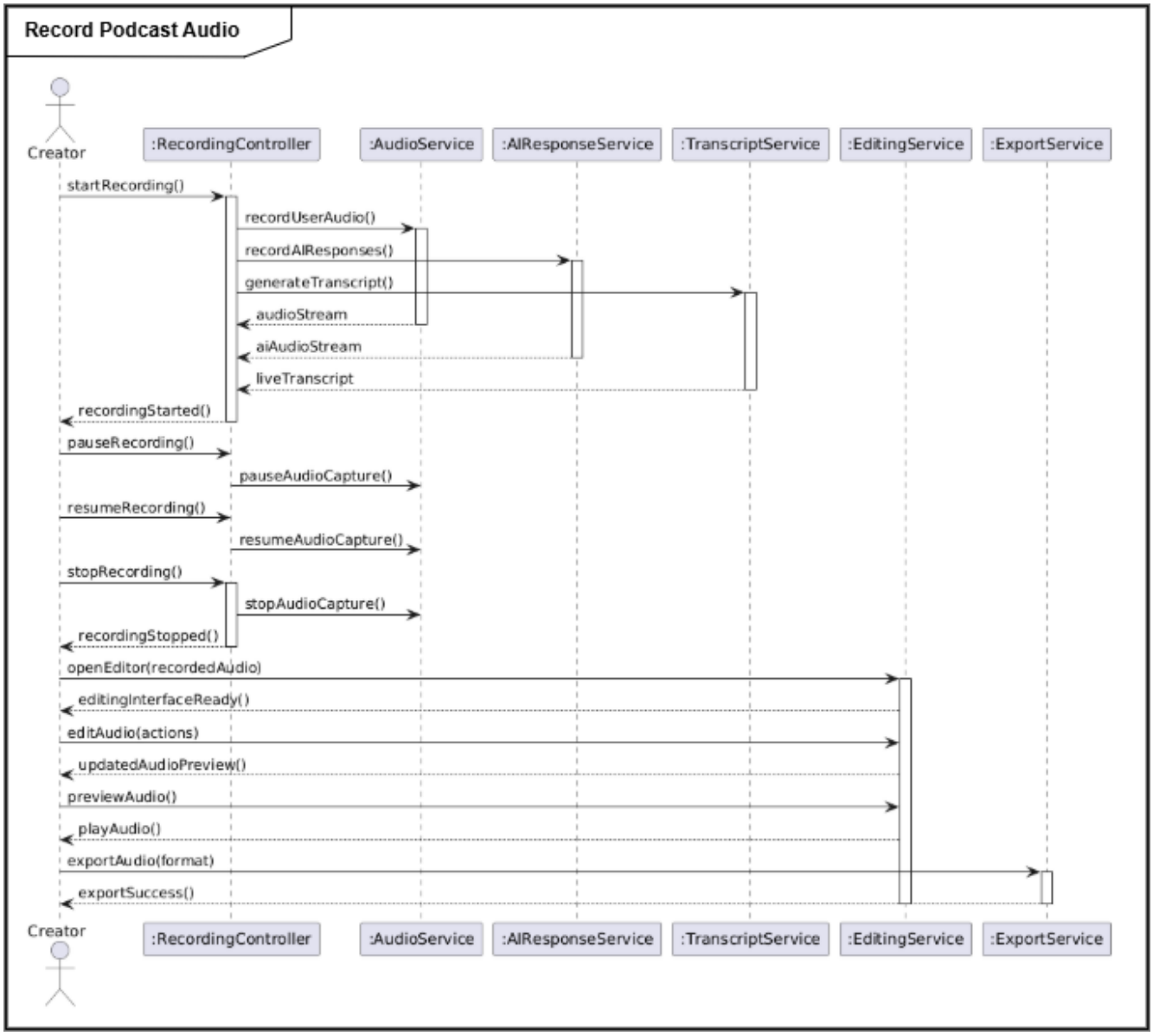


Figure 22: Sequence diagram of module 6

4.3.6 Module 9: Podcast Video Creation and Publishing

Sequence diagram of module 9.

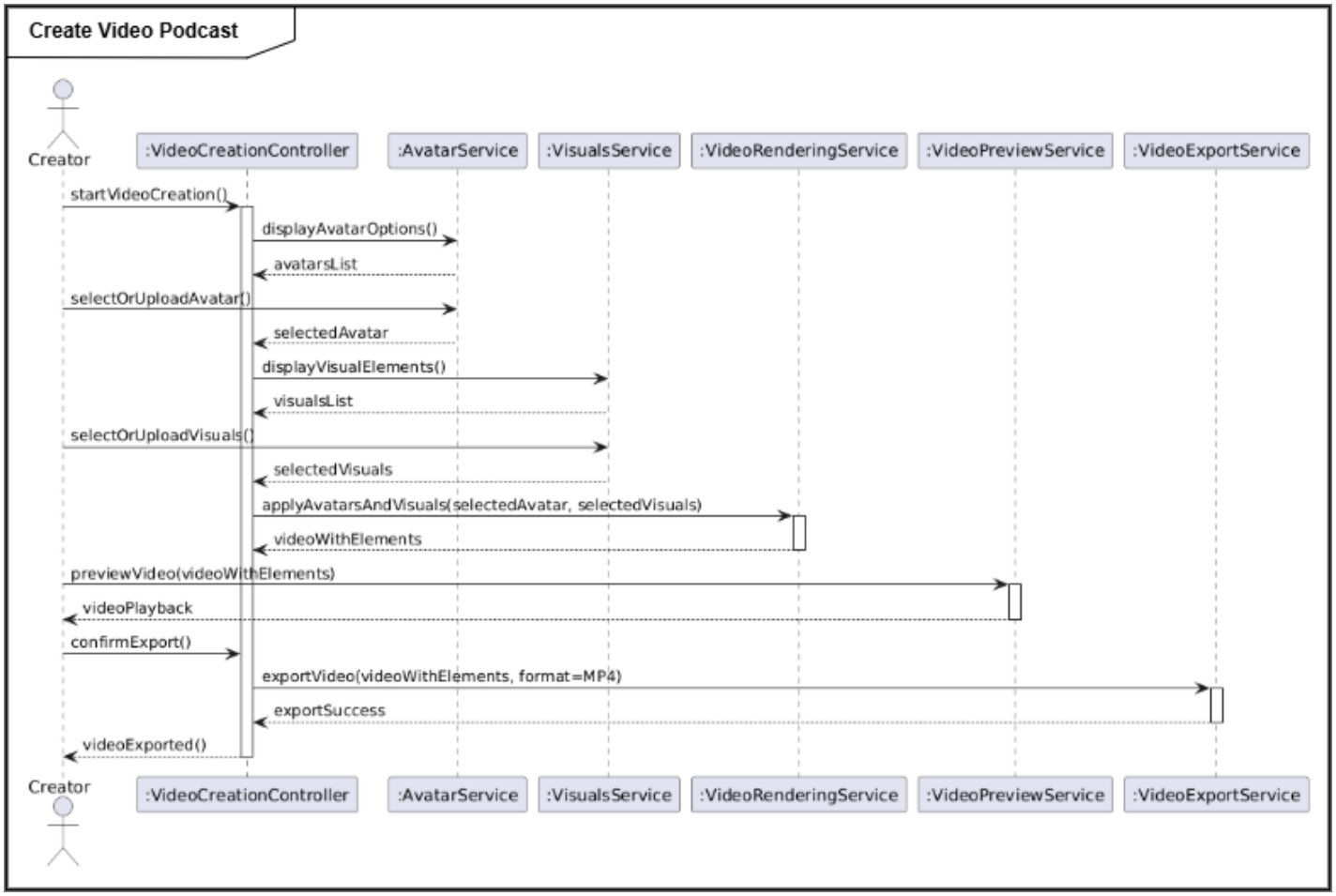


Figure 23: Sequence diagram of module 9

4.3.7 Module 10: Subscription and Payment

Sequence diagram of module 10.

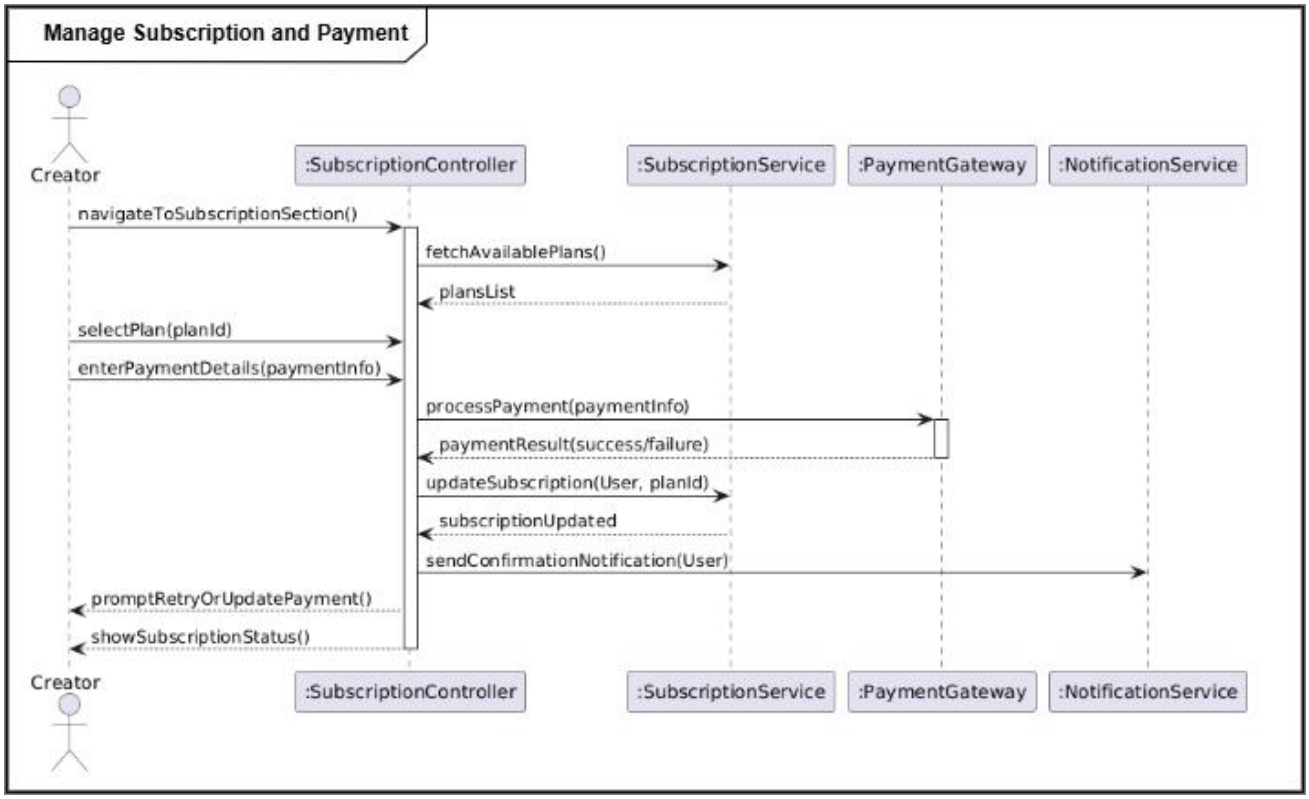


Figure 24: Sequence diagram of module 10

4.3.8 Module 11: FAQs Chatbot

Sequence diagram of module 11.

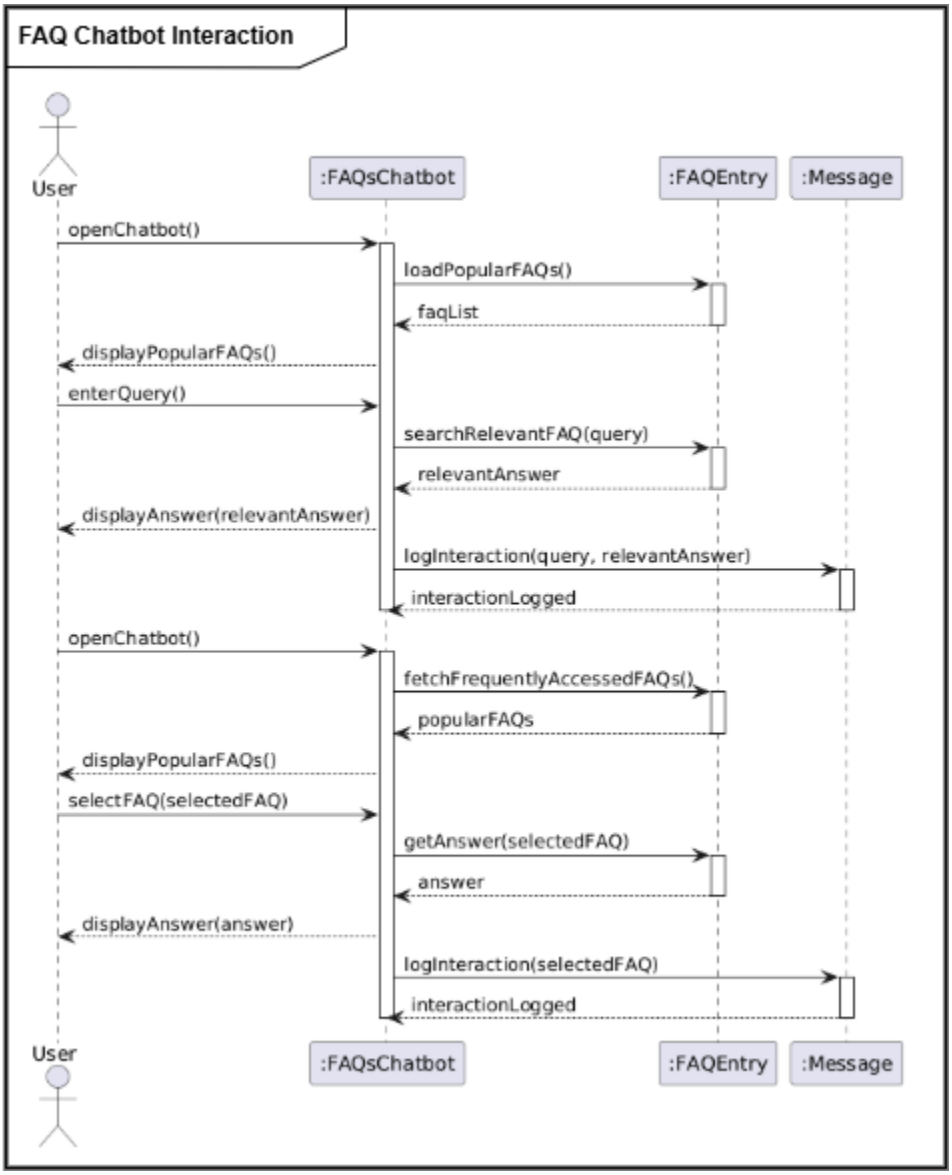


Figure 25: Sequence diagram of module 11

4.3.9 Module 12: Audience Engagement

Sequence diagram of module 12.

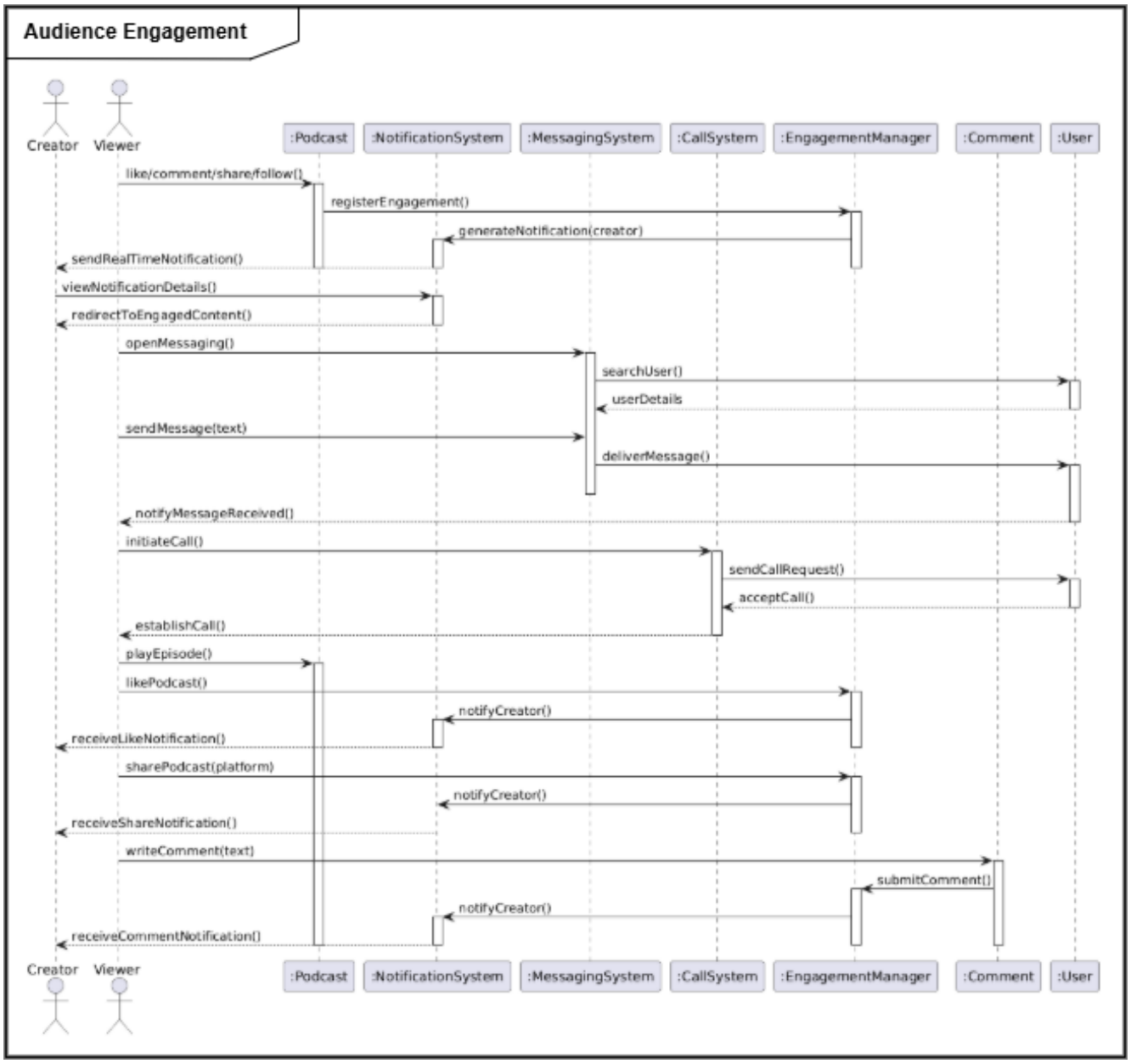


Figure 26: Sequence diagram of module 12

4.3.10 Module 13: Admin Panel and Analytics

Sequence diagram of module 13.

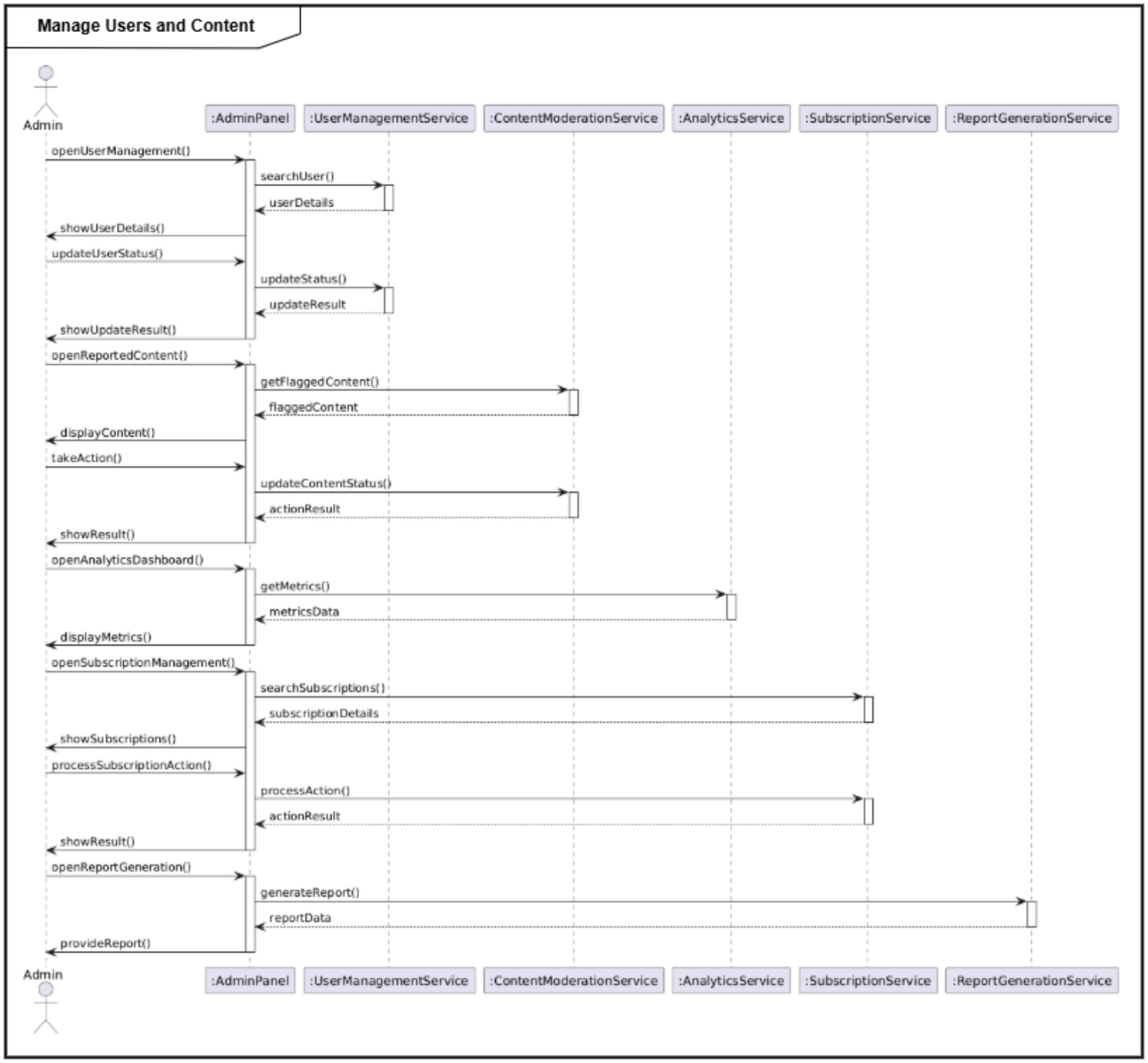


Figure 27: Sequence diagram of module 13

4.4 State Transition Diagrams

The following are the state transition diagrams of the backend processes of our system:

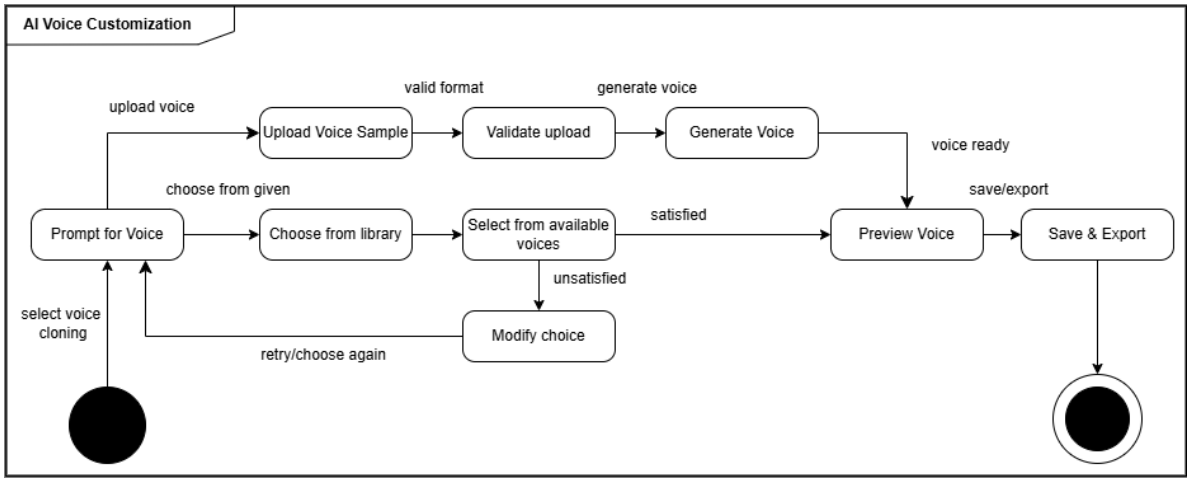


Figure 28: State transition diagram of AI Voice Customization

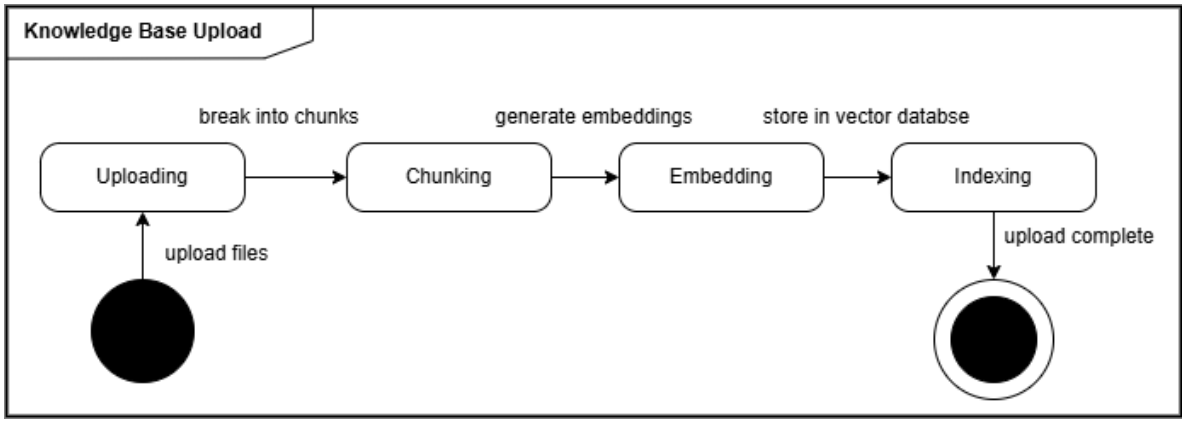


Figure 29: State transition diagram of Knowledge Base Upload

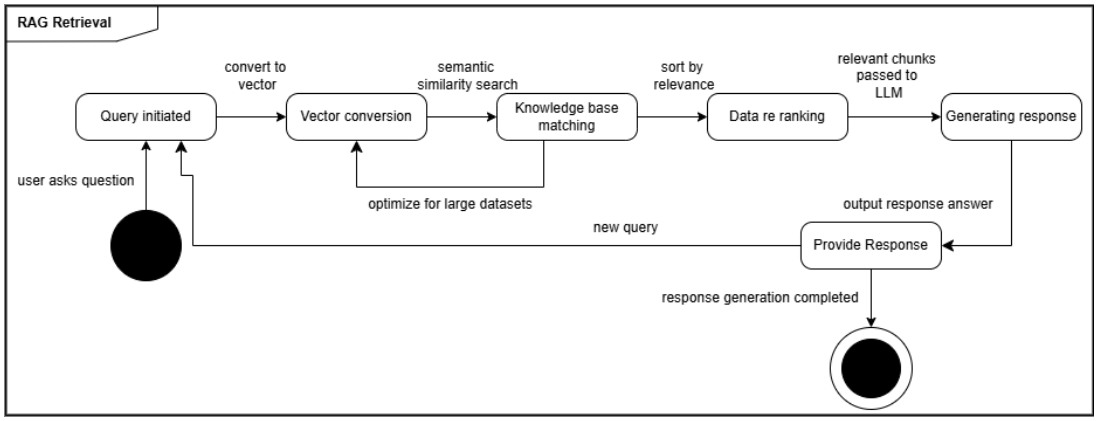


Figure 30: State transition diagram of RAG Retrieval

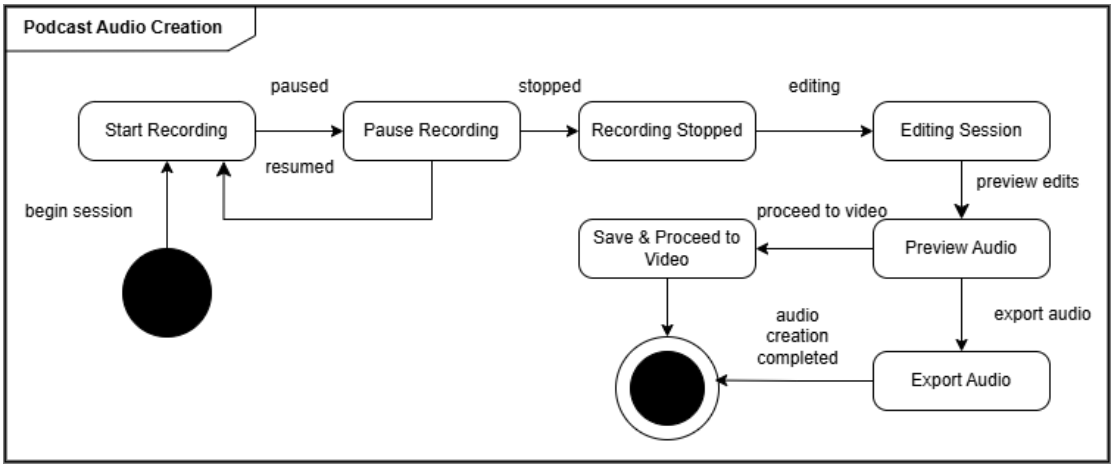


Figure 31: State transition diagram of Podcast Audio Creation

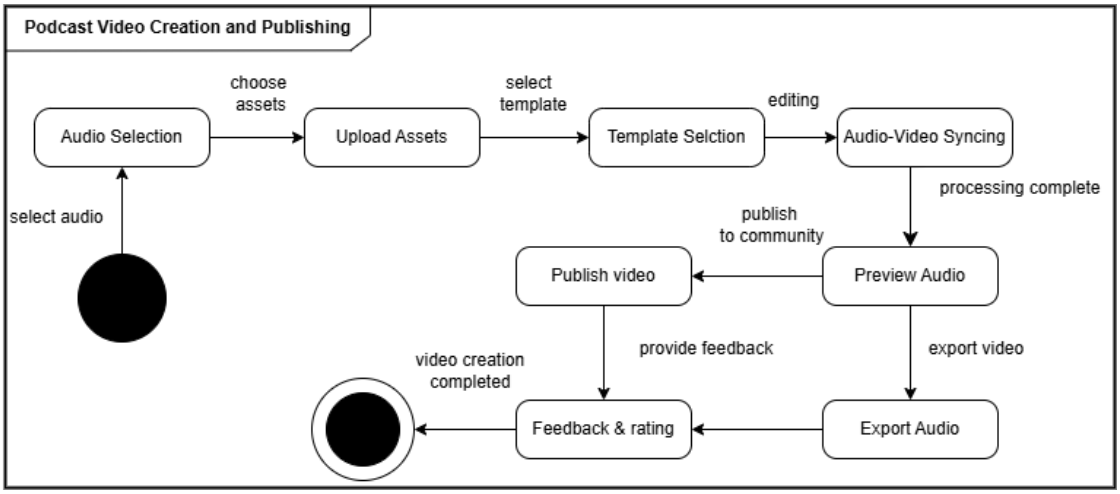


Figure 32: State transition diagram of Podcast Video Creation and Publishing

5. Data Design

The data design of PodVerse is as follows:

5.1 Data Dictionary

Following is the data dictionary of PodVerse:

5.1.1 User Collection

```
{
  userID: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  fullName: { type: String },
  password: { type: String, required: true },
  profilePicUrl: { type: String },
  role: { type: String },
  createdAt: { type: Date },
  bio: { type: String },
  isSubscribed: { type: Boolean }
}
```

Table 1: Data Collection of 'User'

Field Name	Data Type	Constraints	Description
userID	String	Primary Key	Unique identifier for the user
email	String	Unique, Not Null	User's email address
fullName	String	None	User's full name
password	String	Not Null	Hashed password for secure login
profilePicUrl	String	None	URL of the user's profile image
role	String	None	Regular, Admin
createdAt	DateTime	None	Account creation date
bio	String	None	User's biography
isSubscribed	Boolean	None	Subscription status

5.1.2 Podcast Collection

```
{
 podcastID: { type: String, required: true },
 createdBy: { type: String, required: true },
 title: { type: String, required: true },
 description: { type: String },
 category: { type: String },
 audioFilePath: { type: String },
 videoFilePath: { type: String },
 transcript: { type: String },
 coverImageUrl: { type: String },
 createdAt: { type: Date }
}
```

Table 2: Data Collection of 'Podcast'

Field Name	Data Type	Constraints	Description
podcastID	String	Primary Key	Unique identifier for the user
creatorBy	String	Foreign Key	User ID of the podcast creator
title	String	Not Null	Title of the podcast
description	String	None	Description of the podcast
category	String	None	Category/genre of the podcast
audioFilePath	String	None	URL to the podcast audio file
videoFilePath	String	None	URL to the podcast video file
transcript	String	None	Transcript generated during recording
coverImageUrl	String	None	Podcast thumbnail image URL
createdAt	DateTime	None	Date and time the podcast was published

5.1.3 Voice Sample Collection

```
{
 sampleID: { type: String, required: true },
 uploadedBy: { type: String, required: true },
 sampleAudioUrl: { type: String },
 cloneModelID: { type: String },
 createdAt: { type: Date } }
```

Table 3: Data Collection of 'Voice Sample'

Field Name	Data Type	Constraints	Description
sampleID	String	Primary Key	Unique identifier for sample
uploadedBy	String	Foreign Key	Linked user ID
sampleAudioUrl	String	None	URL of uploaded voice sample
cloneModelID	String	None	ID of the trained clone model
createdAt	DateTime	None	Timestamp of creation

5.1.4 Knowledge Base Collection

```
{
 knowledgeID: { type: String, required: true },
 uploadedBy: { type: String, required: true },
 title: { type: String },
 sourceType: { type: String },
 content: { type: String },
 cleanedContent: { type: String },
 vectorID: { type: String },
 uploadedAt: { type: Date }
}
```

Table 4: Data Collection of 'Knowledge Base'

Field Name	Data Type	Constraints	Description
knowledgeID	String	Primary Key	Unique knowledge base ID
uploadedBy	String	Foreign Key	Linked user ID
title	String	None	Title of the knowledge item
sourceType	String	None	PDFs, URLs, CSV, etc.
content	Text	None	Original content
cleanedContent	Text	None	Pre-processed version of the content
vectorID	String	None	Associated vector representation ID

uploadedAt	DateTime	None	Upload timestamp

5.1.5 AI Speaker Collection

```
{
 speakerID: { type: String, required: true },
 userID: { type: String, required: true },
 avatarType: { type: String },
 voiceID: { type: String },
 isHost: { type: Boolean },
 createdAt: { type: Date }
}
```

Table 5: Data Collection of 'AI Speaker'

Field Name	Data Type	Constraints	Description
speakerID	String	Primary Key	Unique ID of the AI Speaker
name	String	Not Null	AI speaker name
language	String	Not Null	Language spoken
voiceStyle	String	Nullable	Style of speaking (e.g., Calm, Energetic)
avatarUrl	String	Nullable	Avatar image or 3D model link
createdAt	DateTime	Not Null	Creation timestamp

5.1.6 Subscription Collection

```
{
 subscriptionID: { type: String, required: true },
 userID: { type: String, required: true },
 planType: { type: String },
 startDate: { type: Date },
 endDate: { type: Date },
 isActive: { type: Boolean }
}
```

Table 6: Data Collection of 'Subscription'

Field Name	Data Type	Constraints	Description
subscriptionID	String	Primary Key	Unique ID of the subscription
UserID	String	Foreign Key	Subscribed user's ID
planType	String	Not Null	Plan (Free, Pro, Premium)
startDate	DateTime	None	Subscription start date
endDate	DateTime	None	Subscription end/renewal date
isActive	Boolean	Default: true	Whether the subscription is currently active

5.1.7 Notification Collection

```
{
 notificationID: { type: String, required: true },
 userID: { type: String, required: true },
 message: { type: String },
 isRead: { type: Boolean },
 sentAt: { type: Date }
}
```

Table 7: Data Collection of 'Notification'

Field Name	Data Type	Constraints	Description
notificationID	String	Primary Key	Unique notification ID
userID	String	Foreign Key	Recipient user ID
title	String	None	Title of the notification
message	String	None	Main content of the notification
createdAt	DateTime	None	Timestamp of the notification
isRead	Boolean	Default: false	Read/unread status

5.1.8 Payment Collection

```
{
 paymentID: { type: String, required: true },
 userID: { type: String, required: true },
 amount: { type: Number },
 paymentDate: { type: Date },
 paymentMethod: { type: String }
}
```

Table 8: Data Collection of 'Payment'

Field Name	Data Type	Constraints	Description
paymentID	String	Primary Key	Unique ID of the payment
userID	String	Foreign Key	User who made the payment
subscriptionID	String	Foreign Key	Related subscription
amount	Float	Not Null	Amount paid
status	String	Not Null	Payment status
paymentDate	DateTime	Not Null	Date and time of payment

5.1.9 FAQs Chatbot Collection

```
{
 faqBotID: { type: String, required: true },
 botName: { type: String },
 description: { type: String },
 createdAt: { type: Date }
}
```

Table 9: Data Collection of 'FAQs Chatbot'

Field Name	Data Type	Constraints	Description
chatbotID	String	Primary Key	Unique chatbot session ID
userID	String	Foreign Key	User who started the session
sessionStart	DateTime	Not Null	When the session began

sessionEnd	DateTime	Nullable	When the session ended

5.1.10 FAQs Entry Collection

```
{
 faqID: { type: String, required: true },
 question: { type: String },
 answer: { type: String },
 associatedBotID: { type: String },
 createdAt: { type: Date }
}
```

Table 10: Data Collection of 'FAQs Entry'

Field Name	Data Type	Constraints	Description
faqID	String	Primary Key	Unique ID of the FAQ entry
question	String	Not Null	FAQ question
answer	Text	Not Null	FAQ answer
category	String	Nullable	Category (e.g., Billing, Technical, General)
createdAt	DateTime	Not Null	Entry creation timestamp

5.1.11 Comment Collection

```
{
 commentID: { type: String, required: true },
 podcastID: { type: String, required: true },
 userID: { type: String },
 text: { type: String },
 postedAt: { type: Date }
}
```

Table 11: Data Collection of 'Comment'

Field Name	Data Type	Constraints	Description
commentID	String	Primary Key	Unique identifier for the comment
userID	String	Foreign Key	User who made the comment

episodeID	String	Foreign Key	Episode the comment is associated with
content	Text	Not Null	The actual comment text
createdAt	DateTime	Not Null	Timestamp when the comment was created
updatedAt	DateTime	Nullable	Timestamp when the comment was last edited

5.1.12 Rating Collection

```
{
 ratingID: { type: String, required: true },
 podcastID: { type: String, required: true },
 userID: { type: String },
 ratingValue: { type: Number },
 ratedAt: { type: Date }
}
```

Table 12: Data Collection of 'Rating'

Field Name	Data Type	Constraints	Description
ratingID	String	Primary Key	Unique identifier for the rating
userID	String	Foreign Key	User who gave the rating
episodeID	String	Foreign Key	Episode that was rated
score	Integer	Not Null	Rating score (e.g., 1 to 5)
comment	String	Nullable	Optional comment/review associated with the rating
createdAt	DateTime	Not Null	When the rating was submitted

5.1.13 Message Collection

```
{
 messageID: { type: String, required: true },
 fromUserID: { type: String, required: true },
 toUserID: { type: String, required: true },
 content: { type: String },
 sentAt: { type: Date },
 isRead: { type: Boolean }
}
```

Table 13: Data Collection of 'Message'

Field Name	Data Type	Constraints	Description
messageID	String	Primary Key	Unique message ID
senderID	String	Foreign Key	User who sent the message
receiverID	String	Foreign Key	User who received the message
content	Text	Not Null	Message text
sentAt	DateTime	Not Null	Timestamp when the message was sent
isRead	Boolean	Default: false	Indicates if the message has been read

6. Implementation

This section contains the implementation details of the system. It includes algorithms, external APIs used by the system, user interface design and deployment details.

6.1 Algorithm

The following are the algorithms used in PodVerse:

<table>
<tr><td colspan="2">Algorithm 1 Registration</td></tr>
<tr><td colspan="2">Input: user_details (email, password)</td></tr>
<tr><td colspan="2">Output: Registration status (success or error message)</td></tr>
<tr><td colspan="2">1. Initialize database_connection
2. Input user_details (email, password, role).
3. Check if role is valid (Applicant or Recruiter):
4. If not valid, return: "Invalid role".
5. Normalize the email to lowercase.
6. Check if the email already exists using checkEmailExists(email):
7. If it exists, return "Email already exists".
8. Encrypt the password using bcrypt (encryptPassword(password)).
9. Save user_details (email and password) to the database.
10. Send a verification email using sendVerificationCode(email).
11. Return "Registration successful. Please verify your email.".</td></tr>
<tr><td>Algorithm 1.1 checkUsernameExists</td><td>Algorithm 1.2: encryptPassword</td></tr>
<tr><td>Input: username</td><td>Input: password</td></tr>
<tr><td>Output: boolean value</td><td>Output: encrypted_password</td></tr>
<tr><td>1: initialize database_connection
2: input email
3: user_record ← query database for user where email = input email
4: if (user_record exists) then
5: return true
6: else
7: return false</td><td>1: initialize encryption_algorithm
2: input password
3: encrypted_password ← encryption_algorithm.encrypt(password)
4: return encrypted_password</td></tr>
<tr><td colspan="2">Algorithm 2 Login</td></tr>
<tr><td colspan="2">Input: email,password</td></tr>
<tr><td colspan="2">Output: Login status (success or error message)</td></tr>
<tr><td colspan="2">1. Initialize database_connection.
2. Input email and password.
3. Normalize the email to lowercase.
4. Query the database for a user with the given email.
5. If no user is found, return "Invalid email".
6. Compare the provided password with the stored password using bcrypt.compare.
7. If the password is incorrect, return "Invalid password".
8. Check if the user is verified (isVerified).
9. If not verified, return "Email not verified".
10. Generate a secure token (setUser(user)).
11. Store the token as a cookie in the response.
12. Return "Login successful".</td></tr>
<tr><td>Algorithm 3 Voice Cloning</td><td>Algorithm 4 TranscribePodcastAudio</td></tr>
<tr><td>Input: User's audio sample (.wav)</td><td>Input: audio_file (.wav)</td></tr>
<tr><td>Output: Cloned voice model ID</td><td>Output: Transcript text</td></tr>
</table>

1: Import necessary libraries (torchaudio, numpy, os, torch) 2: Function preprocess_audio(audio_sample): 3: Convert to mono channel, 16kHz if not already 4: Return cleaned_audio 5: Function extract_embedding(cleaned_audio): 6: Load Tortoise TTS encoder model 7: embedding ← encoder(cleaned_audio) 8: Return embedding 9: Function save_embedding(user_id, embedding): 10: path ← "voice_embeddings/" + user_id + ".pt" 11: Save embedding tensor to path using torch.save() 12: Return path 13: Main Function clone_voice(user_id, audio_sample): 14: cleaned_audio ← preprocess_audio(audio_sample) 15: embedding ← extract_embedding(cleaned_audio) 16: embedding_path ← save_embedding(user_id, embedding) 17: Save embedding_path in MongoDB user profile 18: Return "Voice cloned successfully", embedding_path	1: Import required libraries (torchaudio, os, whisper or other STT model) 2: Function load_stt_model(): 3: model ← load Whisper / Wav2Vec2 model locally 4: Return model 5: Function preprocess_audio(audio_file): 6: Convert audio to 16kHz mono WAV format if needed 7: Return cleaned_audio_path 8: Function transcribe_audio(model, cleaned_audio): 9: transcript ← model.transcribe(cleaned_audio) 10: Return transcript 11: Main Function transcribe_podcast(audio_file): 12: model ← load_stt_model() 13: cleaned_audio ← preprocess_audio(audio_file) 14: transcript ← transcribe_audio(model, cleaned_audio) 15: Save transcript to MongoDB with reference to podcast_id 16: Return transcript
Algorithm 5 GenerateTranscriptSummary	**Algorithm 6 BasicAIUserConversation**
Input: transcript_text	**Input: speaker_turns, conversation_text, user_id**
Output: summary_text	**Output: Generated podcast audio file path**
1: Import HuggingFace transformers library 2: Function load_summarizer_model(): 3: model_name ← "facebook/bart-large-cnn" OR "google/flan-t5-base" 4: tokenizer ← load tokenizer for model_name 5: model ← load pre-trained model from HuggingFace 6: Return tokenizer, model 7: Function preprocess_transcript(transcript_text): 8: Clean special characters and limit to max 1024 tokens 9: Return cleaned_text 10: Function generate_summary(tokenizer, model, input_text): 11: inputs ← tokenizer(input_text, return_tensors="pt", truncation=True) 12: summary_ids ← model.generate(inputs["input_ids"], max_length=150, min_length=40) 13: summary ← tokenizer.decode(summary_ids[0], skip_special_tokens=True) 14: Return summary 15: Main Function summarize_transcript(transcript_text): 16: tokenizer, model ← load_summarizer_model() 17: input_text ← preprocess_transcript(transcript_text) 18: summary ← generate_summary(tokenizer, model, input_text) 19: Save summary in MongoDB linked to transcript_id 20: Return summary	1: Import required libraries (TTS model, pyaudio, ffmpeg, torchaudio) 2: Function synthesize_speaker_audio(text, speaker_embedding): 3: Generate waveform using Tortoise or Tacotron with speaker_embedding 4: Return audio_waveform 5: Function merge_conversation_turns(conversation_list): 6: For each (speaker_role, sentence) in conversation_list: 7: speaker_embedding ← load_embedding(speaker_role) 8: audio ← synthesize_speaker_audio(sentence, speaker_embedding) 9: Append audio to full_session_audio 10: Return full_session_audio 11: Main Function record_ai_podcast(conversation_list, user_id): 12: full_audio ← merge_conversation_turns(conversation_list) 13: file_name ← "user_id_timestamp_ai_session.wav" 14: Save full_audio to Firebase Storage under `podcasts/user_id/` 15: Create podcast metadata (title, timestamp, duration, roles) 16: Save metadata in MongoDB `podcasts` collection 17: Return "Podcast recorded successfully", file path

6.2 External APIs/SDKs

Below are the APIs used in PodVerse :

Table 14: Details of APIs used in the project

Name of API and version	Description of API	Purpose of usage	List down the API endpoint/function/class in which it is used

Stripe (version 2020-08-27)	Credit Card payment integration	Sandbox used for the orders payment	stripe.paymentMethods.create
Cloudinary	Image and Video management solution	Uploading Product Images on Cloudinary server	https://api.cloudinary.com/v1_1/demo/image/upload

6.3 User Interface

6.4 Deployment

7. Testing and Evaluation

This section includes unit testing, functional testing, and integration testing details for the modules we have developed so far. The purpose of unit testing is to ensure that each component of the software functions as intended.

7.1 Unit Testing

This section has the unit level test cases for PodVerse system:

Unit Testing 1: Email Validation Function

Table 15: Unit Test Cases for Email Validation Function

No.	Test Script	Input	Expected Output	Result
1	validateEmailFormat	Email: abc@gmail.com	true	Pass
2	validateEmailFormat	Email: abc.gmail.com	false	Pass
3	validateEmailFormat	Email: `` (empty)	false	Pass

Unit Testing 2: checkEmailExists()

Table 16: Unit Test Cases to Check Email Exists

No.	Test Case/Test Script	Attribute and Value	Expected Result	Result
1	checkEmailExists	Email: **sadiaimran837@gmail.com**, Password: Sadia@123	true	Pass
2	checkEmailExists	Email: **sadiaimran837@gmail.com**	Error: "Email already exists"	Pass

Unit Testing 3: Password Encryption & Verification

Table 17: Unit Test Cases for Password Encryption & Verification

No.	Test Script	Input	Expected Output	Result
1	encryptPassword()	Input: Test123!	Encrypted string returned	Pass
2	verifyPassword()	Input: Test123!, Hash	true	Pass
3	verifyPassword()	Input: WrongPass, Hash	false	Pass

Unit Testing 4: OTP Generator & Validator

Table 18: Unit Test Cases for OTP Generator & Validator

No.	Test Script	Input	Expected Output	Result
1	generateOTP()	Triggered	6-digit numeric OTP	Pass
2	verifyOTP()	Input: 123456, Match	success	Pass
3	verifyOTP()	Input: 999999, Mismatch	failure	Pass

Unit Testing 5: Voice Cloning File

Table 19: Unit Test Cases for Voice Cloning File

No.	Test Script	Input	Expected Output	Result
1	processVoiceSample()	File: voice.wav	Cleaned mono audio	Pass
2	extractVoiceProfile()	Cleaned audio	Voice profile (speaker vector) generated	Pass
3	saveVoiceProfile()	Profile data, user ID	Voice profile linked to user in database	Pass

Unit Testing 6: Summarization

Table 20: Unit Test Cases for Summarization

No.	Test Script	Input	Expected Output	Result
1	generateSummary()	Text: 1000 words	Summary ≤ 150 words	Pass
2	generateSummary()	Input: `` (empty)	Error: Missing transcript	Pass
3	saveSummaryToDB()	Summary, transcript_id	MongoDB write success	Pass

Unit Testing 7: Transcription Module

Table 21: Unit Test Cases for Transcription Module

No.	Test Case	Attribute and Value	Expected Result	Result
1	preprocessAudio()	`.wav` file	Cleaned 16kHz audio path	Pass
2	transcribeAudio()	Cleaned audio	Transcript string	Pass
3	saveTranscriptToDB()	Transcript, podcast_id	MongoDB write success	Pass

Unit Testing 8: AI-Conversation Reply Generator

Table 22: Unit Test Cases for AI-Conversation Reply Generator

No.	Test Script	Input	Expected Output	Result
1	generateReplyFromText()	Transcript: "Tell me something"	String reply from AI	Pass
2	saveToConversationLog()	user_id, AI reply	DB write success	Pass

Unit Testing 9: AI Speaker Customization

Table 23: Unit Test Cases for AI Speaker Customization

No.	Test Case/Test Script	Attribute and Value	Expected Result	Result
1	Upload valid avatar image	File: avatar.png	Image preview displayed, file saved	Pass
2	Upload unsupported avatar file	File: avatar.txt	Error: "Only PNG, JPG supported"	Pass
3	Select speaker role	Option: AI Guest	Role saved in session	Pass
4	Upload valid knowledge base	File: kb_text.pdf	Text extracted and saved	Pass
5	Upload unsupported knowledge base	File: kb.exe	Error: "Invalid file format"	Pass
6	Save speaker config	Role + Avatar + KB	MongoDB profile updated	Pass

Unit Testing 10: Audio Recorder Module

Table 24: Unit Test Cases for Audio Recorder Module

No.	Test Script	Input	Expected Output	Result
1	startRecording()	Trigger	Mic access granted	Pass
2	stopRecording()	File path	Audio saved	Pass
3	uploadToFirebase()	File: audio.wav	URL returned	Pass

Unit Testing 11: Knowledge Base Processing

Table 25: Unit Test Cases for Knowledge Base Processing

No.	Test Case/Test Script	Attribute and Value	Expected Result	Result
1	Upload valid .txt file	File: facts.txt	Text content extracted and saved	Pass
2	Upload valid .pdf file	File: data.pdf	Text extracted from PDF	Pass
3	Upload corrupted file	File: bad.pdf	Error: "Unable to read file content"	Pass
4	Link content to speaker config	Speaker ID: ai_6123abc	MongoDB updated with knowledge source	Pass

7.2 Functional Testing

The following are the functional test cases of PodVerse:

Functional Testing 1: SignUp & OTP Verification

Table 25: Functional Test Cases for SignUp & OTP Verification

No.	Test Case/Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	Register with valid details	Email: sadiaimran837@gmail.com, Password: Sadia@123	OTP sent to registered email	OTP screen appeared, email received	Pass
2	Duplicate email	Email: sadiaimran837@gmail.com	Error: "Email already in use"	Error message displayed	Pass

Functional Testing 2: Login

Table 26: Functional Test Cases for Login

No.	Test Case/Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	Valid login credentials	Email: rabeeachughtai1@gmail.com, Password: Rabeea@123	Dashboard loaded	User redirected to dashboard	Pass

2	Incorrect password	Email: rabeeachughtai1@gmail.com, Password: Wrong123	Error: "Invalid credentials"	Error toast displayed	Pass

Functional Testing 3: Podcast Transcription

Table 27: Functional Test Cases for Podcast Transcription

No.	Test Case/Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	Upload podcast audio	File: podcast.wav	Full transcript displayed on screen	Transcript text loaded in textarea	Pass
2	Upload empty or silent file	File: silence.wav	Error: “No speech detected”	Warning shown in toast	Pass

Functional Testing 4: Voice Cloning

Table 28: Functional Test Cases for Voice Cloning

No.	Test Case/Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	Upload valid voice sample	File: user_voice.wav	Voice cloned successfully, stored for reuse	“Voice cloned” message shown	Pass
2	Upload unsupported format	File: user_voice.mp3	Error: “Only WAV files are supported”	Validation error appeared	Pass

Functional Testing 5: Transcript Summarization

Table 29: Functional Test Cases for Transcript Summarization

No.	Test Case/Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	Generate summary	Transcript: 1000+ words	Summary displayed (≤150 words)	Short summary shown in summary tab	Pass

2	Click summarize with empty input	Transcript: (empty)	Error: "Transcript required"	Validation toast: “Transcript empty”	Pass

Functional Testing 6: AI-User Conversation

Table 30: Functional Test Cases for AI-User Conversation

No.	Test Case/Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	Upload mic prompt	File: user_prompt.wav	AI reply generated and shown	Reply displayed below waveform	Pass
2	Submit empty prompt	File: silence.wav	Error: “Please speak to continue”	“Please speak” toast shown	Pass

Functional Testing 7: Podcast Audio Recording & Upload

Table 31: Functional Test Cases for Podcast Audio Recording & Upload

No.	Test Case/Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	Start/Stop recording	Record → Stop clicked	Recording preview generated	Waveform rendered	Pass
2	Upload to Firebase	File: recorded_audio.wav	File stored, Firebase URL returned	Link received and saved	Pass
3	Save metadata	Title: “Ep1”, Tags: [AI, Tech]	Podcast entry created in MongoDB	Metadata saved, confirmation shown	Pass

Functional Testing 8: Podcast Library Access

Table 32: Functional Test Cases for Podcast Library Access

No.	Test Case/Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	Load personal podcast list	User ID: rabeeachughtai1@gmail.com	User podcasts listed	Podcast cards shown on screen	Pass
2	Play uploaded podcast	File URL from DB	Audio plays in embedded player	Media player started with audio	Pass
3	Show sample podcasts only	Filter: sample=true	Sample podcasts displayed	Filtered list of samples shown	Pass

Functional Testing 9: AI Speaker Customization

Table 33: Functional Test Cases for AI Speaker Customization

No.	Test Case/Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	Upload avatar image	File: avatar.png	Avatar displayed and saved	Image shown on profile	Pass
2	Choose speaker role	Role: AI Host	Role stored successfully	Display updated with AI Host tag	Pass
3	Upload valid knowledge base	File: kb_notes.pdf	Text extracted and attached to AI speaker	Content linked and displayed	Pass
4	Upload unsupported KB	File: kb.exe	Error: “Invalid file format”	Error toast: "Only PDF/TXT allowed"	Pass

7.3 Business Rules Testing

This section contains various decisions and actions according to the business rules.

Business Rules Testing 1: Only one account can be registered for a given email address

Table 34: Business Rules Testing 1

Conditions	**Rule 1**	**Rule 2**	**Rule 3**
Email already registered	Yes	No	No
Password valid	Yes	Yes	No
Actions			
Account created	No	Yes	No

Business Rules Testing 2: Password must meet security criteria

Table 35: Business Rules Testing 2

Conditions	**Rule 1**	**Rule 2**	**Rule 3**
Password contains digits	Yes	No	No
Password contains letters	Yes	Yes	No
Actions			
Account created	Yes	No	No

Business Rules Testing 3: Voice Cloning – Audio Upload Constraints

Table 36: Business Rules Testing 3

Conditions	**Rule 1**	**Rule 2**
File format supported	Yes	No
Audio meets quality standards	Yes	Yes
Audio accepted	Yes	No
File format supported	Yes	No

Business Rules Testing 4: Knowledge Upload Format

Table 37: Business Rules Testing 4

Conditions	Rule 1	Rule 2
File format supported	Yes	No
Data is structured	Yes	Yes
File accepted	Yes	No

7.4 Integration Testing

This section has the integration test cases for the modules that we have developed. They make sure that the connections and interfaces between various components of the system function correctly.

Integration Testing 1: User Registration & Email Verification

Table 38: Integration Testing 1 for User Registration & Email Verification

No.	Test case/ Test script	Attribute and value	Expected result	Actual result	Result
1	Valid Registration	Email, Password, Role = "Applicant"	"Registration successful. Please verify your email."	Same	Pass
2	Duplicate Email	Already existing email	"Email already exists"	Same	Pass
3	Invalid Role	Role = "Manager"	"Invalid role"	Same	Pass

Integration Testing 2: Login with Verification Flow

Table 39: Integration Testing 2 for Login with Verification Flow

No.	Test case/ Test script	Attribute and value	Expected result	Actual result	Result
1	Login with verified user	Email, Password	"Login successful"	Same	Pass
2	Login with unverified user	Valid credentials	"Email not verified"	Same	Pass
3	Incorrect password	Valid email, wrong password	"Invalid password"	Same	Pass

Integration Testing 3: Voice Cloning Flow

Table 40: Integration Testing 3 for Voice Cloning Flow

No.	Test case/ Test script	Attribute and value	Expected result	Actual result	Result
1	Clone voice with valid .wav	UserID, audio_sample	"Voice cloned successfully", path	Same	Pass
2	Upload unsupported format	.mp3 file	Error: "Unsupported format"	Same	Pass

Integration Testing 4: Podcast Transcription

Table 41: Integration Testing 4 for Podcast Transcription

No.	Test case/ Test script	Attribute and value	Expected result	Actual result	Result
1	Transcribe valid podcast	Valid .wav file	Transcript returned and saved	Same	Pass
2	Upload non-WAV file	.mp3	Error: Invalid format or convert	Same	Pass

Integration Testing 5: Transcript Summarization

Table 42: Integration Testing 5 for Transcript Summarization

No.	Test case/ Test script	Attribute and value	Expected result	Actual result	Result
1	Summarize valid transcript	1024 token input	Summary saved in MongoDB	Same	Pass
2	Excessively long input	2000+ tokens	Summarized within token limits	Same	Pass

Integration Testing 6: AI Podcast Generation

Table 43: Integration Testing 6 for AI Podcast Generation

No.	Test case/ Test script	Attribute and value	Expected result	Actual result	Result
1	Record podcast with 2 speakers	conversation_list	Podcast saved in Firebase, metadata stored	Same	Pass
2	Missing speaker embedding	Unknown role	Error or fallback voice	Same	Pass

8. References

1. P. Nave. “Exploring RAG: Essential Steps to Implementing AI Agents and Deep Learning Strategies.” Internet: https://medium.com/@pratikabnave97/exploring-rag-essential-steps-to-implementing-ai-agents-and-deep-learning-strategies-f41ceda143c4, Mar. 10, 2024 [Nov. 10, 2024].
2. S. Smith. “The Future of Podcasting in 2024 and Beyond: Key Predictions.” Internet: https://www.opus.pro/blog/the-future-of-podcasting-in-2024-and-beyond-key-predictions, Mar. 22, 2024 [Nov. 10, 2024].
3. G. Harris. “AI Knowledge Base: Enhancing Intelligence for Smart Decisions.” Internet: https://www.getguru.com/reference/ai-knowledge-base, Apr. 5, 2024 [Nov. 10, 2024].